学校代码： 10264
研究生学号： M210811357

上 海 海 洋 大 学

硕士学位论文

题　　目： 基于深度强化学习的机械臂轨迹规划研究

英文题目： Research On Robotic Arm Trajectory Planning Based On Deep Reinforcement Learning

研究方向： 强化学习

姓　　名： 周素成

指导教师： 匡兴红

二 O 二四年 五 月 九 日

上海海洋大学 硕士学位论文
答辩委员会成员名单

姓名	工作单位	职称	备注
张颖	上海海事大学	教授	主席
吕春峰	上海海洋大学	副教授	委员
朱建平	上海海洋大学	副教授	委员
赵煜	上海海洋大学	讲师	秘书
答辩地点	工程学院 113	答辩日期	2024.5.9

基于深度强化学习的机械臂轨迹规划研究

摘　要

随着科技进步和工业生产的发展，机械臂的应用愈发广泛。其中复杂环境下的机械臂轨迹规划算法研究成为研究热点。传统机械臂轨迹规划算法对环境要求高，无法适应环境的变化且需要在静态环境下预先规划好轨迹。随着智能化的进步，深度强化学习算法实现机械臂的轨迹规划和路径规划成为一种可能。目前国内外对于深度强化学习在轨迹规划上的研究也有一些研究成果。因此，本文对复杂环境下机械臂的轨迹规划展开研究，对机械臂的国内外研究、强化学习算法在机械臂上的应用进行总结，针对非结构化的动态环境提出深度强化学习实现机械臂自主轨迹规划的研究方法。主要研究内容如下：

首先，根据 Franka 机械臂的 D-H 参数分析的运动学原理，推导了运动学表达式。开发了基于 PyBullet 的物理仿真环境，并搭建了强化学习算法平台。使用蒙特卡洛方法计算分析了工作空间的范围并完成了对机械臂状态空间、动作空间和奖励函数的设计。针对动态环境下的机械臂轨迹规划中，机械臂无法避开移动障碍物的同时，无法快速追踪到目标点的问题，研究了融合欧氏距离奖励函数、二值化奖励函数和实时指导奖励函数的加权组合奖励函数。

接着，选择使用主流算法近端策略优化算法（Proximal Policy Optimization, PPO）和软演员-评论家算法（Soft Actor-Critic, SAC）对改进奖励函数和原奖励函数进行实验评估，对两种基础算法进行参数调节后开展实验。在两种不同难度的场景下，比较两种算法模型所获得的成功率、平均奖励值、安全率，验证了改进奖励函数的效果。并选择性能较优的 SAC 算法作为基准算法，并将改进奖励函数作为后续实验的基准奖励函数。

然后，针对复杂环境下机械臂难以适应外界环境，导致机械臂自主轨迹规划效果不稳定，收敛速度慢的问题，提出了一种改进的 SAC 算法（AL-PER-SAC）。在 SAC 算法的策略网络和评价网络中引入结合注意力机制（Attention Mechanism）和长短期记忆神经网络（Long Short-Term Memory，LSTM）的 Attn-LSTM 结构，通过调整注意力对状态信息实现重要性分配，使用 LSTM 网络结合历史状态和当前信息，使得机械臂充分利用复杂的状态信息并作出更佳的决策。结合优先级经验回放机制（Prioritized Experience Replay, PER），通过采集样本价值较高的数据，

加快机械臂训练的收敛速度和自主轨迹规划的效率。

最后，在 PyBullet 物理仿真环境下，在距离阈值为 0.1 和 0.05 的不同场景和测试场景下进行实验，对成功率、奖励值、安全展开分析。实验结果表明使用融合 Attn-LSTM 和优先级经验采样的 SAC 算法在复杂动态环境中有更高的自主规划的成功率和收敛速度，同时安全性得到提高。此外，在难度更高的场景下进行实验，通过对机械臂末端执行器与目标点的距离和机械臂本体到障碍物的最短距离展开分析，实验结果表明 AL-PER-SAC 算法实现自主轨迹规划的稳定性和可靠性最高。

关键词：不确定性环境，强化学习算法，轨迹规划，奖励函数

Research On Robotic Arm Trajectory Planning Based On Deep Reinforcement Learning

ABSTRACT

With the advancement of technology and the development of industrial production, the application of robotic arms is becoming increasingly widespread. The research on trajectory planning algorithms for robotic arms in complex environments has become a hot topic. Traditional robotic arm trajectory planning algorithms have high environmental requirements, cannot adapt to changes in the environment, and require pre planning of trajectories in static environments. With the advancement of intelligence, deep reinforcement learning algorithms have become possible for trajectory planning and path planning of robotic arms. At present, there are also some research results on the application of deep reinforcement learning in trajectory planning both domestically and internationally. Therefore, this article conducts research on trajectory planning of robotic arms in complex environments, summarizes domestic and foreign research on robotic arms, and summarizes the application of reinforcement learning algorithms on robotic arms. A research method for deep reinforcement learning to achieve autonomous trajectory planning of robotic arms is proposed for unstructured dynamic environments. The main research content is as follows:

Firstly, based on the kinematic principle of the D-H parameter analysis of the Franka robotic arm, the kinematic expression was derived. Developed a physical simulation environment based on PyBullet and built a reinforcement learning algorithm platform. The Monte Carlo method was used to calculate and analyze the scope of the workspace, and the design of the state space, action space, and reward function of the robotic arm was completed. Aiming at the problem of the robot arm being unable to avoid moving obstacles and quickly track the target point in dynamic environment trajectory planning, a weighted combination reward function integrating Euclidean distance reward function, binary reward function, and real-time guidance reward function was studied.

Next, we chose to use mainstream algorithms such as Proximal Policy Optimization (PPO) and Soft Actor Critic (SAC) to experimentally evaluate the

improved reward function and the original reward function. After adjusting the parameters of the two basic algorithms, we conducted experiments. The success rate, average reward value, and safety rate of the two algorithm models were compared in two different difficulty scenarios, and the effectiveness of the improved reward function was verified. And select the SAC algorithm with better performance as the benchmark algorithm, and use the improved reward function as the benchmark reward function for subsequent experiments.

Then, an improved SAC algorithm (AL-PER-SAC) was proposed to address the problem of the robotic arm's difficulty in adapting to external environments in complex environments, resulting in unstable autonomous trajectory planning and slow convergence speed. Introducing Attn-LSTM structure that combines attention mechanism and Long Short Term Memory (LSTM) neural network into the strategy network and evaluation network of the SAC algorithm. By adjusting attention to state information, importance allocation is achieved. The LSTM network combines historical and current information, allowing the robotic arm to fully utilize complex state information and make better decisions. By combining the Prioritized Experience Replay (PER) mechanism and collecting data with high sample value, the convergence speed of robotic arm training and the efficiency of autonomous trajectory planning are accelerated.

Finally, in the PyBullet physical simulation environment, experiments were conducted in different scenarios and testing scenarios with distance thresholds of 0.1 and 0.05 to analyze the success rate, reward value, and security. The experimental results show that the SAC algorithm using the fusion of Attn-LSTM and priority empirical sampling has a higher success rate and convergence speed in autonomous planning in complex dynamic environments, while improving security. In addition, experiments were conducted in more challenging scenarios. By analyzing the distance between the end effector of the robotic arm and the target point, as well as the shortest distance from the robotic arm to the obstacle, the experimental results showed that the AL-PER-SAC has higher stability in achieving autonomous trajectory planning.

KEY WORDS: Uncertain environments, reinforcement learning algorithms, trajectory planning, reward functions

目录

第 1 章 绪论 1
1.1 研究背景及意义 1
1.2 国内外研究现状 1
1.2.1 机器人轨迹规划研究现状 1
1.2.2 强化学习的发展 3
1.2.3 深度强化学习奖励机制的研究现状 4
1.2.4 深度强化学习在机械臂控制中的研究 5
1.3 主要研究内容和章节安排 7
1.3.1 主要内容 7
1.3.2 章节安排 7
第 2 章 相关理论概述 9
2.1 引言 9
2.2 机械臂位姿变换 9
2.2.1 机械臂的姿态描述 9
2.2.2 机械臂坐标系变换 10
2.3 马尔可夫决策 12
2.4 深度强化学习算法 13
2.4.1 基于值函数的强化学习算法 13
2.4.2 基于演员－评论家的深度强化学习算法 13
2.4.3 基于策略的深度强化学习算法 14
2.5 本章小结 16
第 3 章 基于深度强化学习的机械臂轨迹规划算法建模及改进 17
3.1 引言 17
3.2 机械臂运动学分析 17
3.3 仿真实验环境搭建 20
3.4 算法模型构造 22
3.4.1 状态空间和行为空间的设计 22
3.4.2 奖励函数的设计 23
3.4.3 奖励函数的改进 24

3.5 模型验证实验 25
3.5.1 实验参数 26
3.5.2 实验 1 分析 27
3.5.3 实验 2 分析 29
3.6 本章小结 32
第 4 章 改进 SAC 算法的机械臂自主轨迹规划 33
4.1 引言 33
4.2 注意力机制 33
4.3 长短期记忆神经网络 34
4.4 SAC 算法的改进 35
4.4.1 Attn-LSTM 模块的引入 35
4.4.2 优先级经验回放 36
4.5 机械臂自主轨迹规划流程 38
4.6 实验参数设置 40
4.7 实验结果分析 41
4.7.1 环境 1 对比实验 42
4.7.2 环境 2 对比试验 44
4.7.3 距离实验 47
4.8 本章小结 48
第 5 章 总结与展望 50
5.1 全文总结 50
5.2 后续工作展望 50
参考文献 52
致 谢 57
攻读学位期间的研究成果 58

第 1 章 绪论

1.1 研究背景及意义

得益于科技和生产力的发展，机械臂广泛应用于生产生活中，并帮助人类更好得适应恶劣环境下的工作。机械臂的路径和轨迹优劣程度对任务完成得好坏也有直接影响。传统机械臂控制技术通常采用运动控制或者力位控制。工业机械臂的控制通常是由技术人员根据任务需求，通过示教模式设计机械臂的末端运动轨迹。这种机械臂轨迹控制的效率高，但限制于指定的工作空间内，人与机器存在一定的距离。随着工业环境的日益复杂，传统机械臂控制已经难以适应复杂环境。传统机械臂控制技术的优势是控制稳定，不会发生过多的意外，但其轨迹规划的误差可能会因复杂环境造成而影响。因此机械臂的轨迹优化问题成为研究热点。

人工智能技术的出现，使得机械臂自主进行运动控制成为可能。深度强化学习的出现，使得人工智能技术应用于游戏、智能驾驶、机器人控制等方面。深度强化学习拥有环境感知能力和强化学习的决策力，通过与环境持续交互，以试错的方式训练机械臂，寻找控制的最佳策略。深度强化学习的输入来自于外界环境，例如与视觉、多传感技术结合，借助深度学习技术处理图片、视频等信息，提取有效特征作为强化学习的输入[1]。因此深度强化学习具备了类人类处理问题的思维和方法。面对复杂环境下的机械臂自主轨迹规划问题，深度强化学习展现出巨大的优势和前景。

1.2 国内外研究现状

1.2.1 机器人轨迹规划研究现状

目前研究较为详细的为机械臂和移动机器人的轨迹规划研究。机械臂轨迹规划跟路径规划不同的是，它需要考虑规划的效率，路径的长短以及规划的平滑性等。轨迹规划方法分为一般和最优的轨迹规划。一般方法分别为笛卡尔空间坐标系和关节空间坐标系下的轨迹规划算法。关节空间下的轨迹规划方法是用机械臂末端的物理变量，比如速度，加速度和位置来表示，对应方法有多项式插值法和 B 样条曲线。笛卡尔空间下的方法是直线和圆弧等方法。最优方法包括三大类，分别为能量、时间和冲击最优。其中时间最优方法有三次 B 样条曲线方法，能量最优方法有拉格朗日插值法，而冲击最优方法有优化加速度、聚类算法和粒子群算法（Particle Swarm Sptimization, PSO）。常用的智能优化算法可以提高轨迹优化能力，包括遗传算法、快速扩展随机数法等。接下来对国内外相关研究展开论述。

左国玉[2]针对机械臂冲击轨迹规划问题，提出基于改进多目标粒子群算法，使用 5 次 B 样条曲线对路径点插值，获得机械臂物理量连续的运动轨迹。并将机械臂关节运动学约束作为优化问题的约束条件，使用变异算子、自适应调参方式优化目标函数。实验验证 5 次 B 样条曲线插值能满足关节平滑和连续性的要求，而改进的算法可以获得较好的 Pareto 前沿面，对机械臂的稳定性和效率有明显提高。刘福深等人[3]针对机器人由于抖动问题导致精度不足的问题，提出基于改进型 Sigmoid 的高阶滑膜控制器。采用零点连续的 sigmoid 函数代替开关函数。实验结果表明改进算法减少系统在滑膜态高频切换引起的抖震，提高了稳定性。

陈师宽等人[4]针对清理变电站变压器空间过于狭窄的问题，设计了一款冗余机械臂，为了缩短轨迹规划的时间，减少随机数目，提出改进 RRT 算法和贪心算法进行轨迹规划。并利用贝塞尔曲线对轨迹平滑处理，奖励轨迹路线曲率。实验证明，改进后的 RRT 算法轨迹更精确、曲率小。Guanglei Wu 等人[5]针对最小化执行时间、能耗和关节抖动等问题，提出了一种优化机器人性能的轨迹规划方法，采用量子边曲线拟合轨迹，通过求解多目标优化问题，获得最优曲线参数和分布式曲线段执行时间，用机械臂对其轨迹规划技术进行了数值说明，并通过实验比较了执行时间和无变量联合动态优化，表明了该方法的有效性。Di Yang 等人[6]针对在高速往复运动过程中，高速并行机器人的非线性动态特性对系统的动态响应和稳定性问题，提出了一种考虑速度约束的等高挑选轨迹规划方法，利用反向传播（BP）神经网络实现在线轨迹优化，以提高高速并行机器人的 PAP 能力和稳定性。实验结果表明改进方法有利于高速并行机器人的应用。Phelipe W. Oliveira 等人[7]考虑到用于机器人操纵器的轨迹规划最小挺举原理用于机器人追踪的缺点，提出一个全面的框架，用于跟踪最小挺举（MJ）机器人轨迹的类 PID 控制器增益的最优估计。还利用粒子群优化算法，同时为所有关节的控制器的增益寻找最优值。所提方法经过实验验证，表明具有较高稳定性。

Chengjun Wang 等人[8]针对铸造排序机械臂的效率和平滑轨迹问题，提出了一种时间最优轨迹规划方法，该使用了成吉思汗鲨鱼（GKS）行为启发和分段插值多项式，采用分割方法将三次和五次多项式相结合，求解未知参数用于规划轨迹。同时又引入了动态非线性学习因子来提高轨迹在关节空间中的平滑性和效率，经过实验仿真，结果表明优化后的机械臂实现了平滑、连续的轨迹速度。Zesheng Wang[9]针对类人手臂技术难以实现长期目标的问题，提出一种七自由度串行并行混合人性机械臂。为了避免关节角的突变，提出了一个考虑行程时间、能耗和转矩波动的多优化模型来规划五次 b 样条插值的轨迹。采用精英非主导分类遗传算法（NSGA-II）对模型进行处理，通过定义目标的权值来确定最优解。并结合平

衡和变形协调方程，得到动力学模型。实验表明该方法具有较好的效果。以上研究人员在传统机械臂轨迹规划研究方法的基础上，使用粒子群算法、样条插值、贝塞尔曲线平滑等方法对机械臂的轨迹进行优化。同时结合神经网络技术通过智能化技术的处理，实现对在线轨迹的优化。

1.2.2 强化学习的发展

强化学习是在最优控制和机器学习和心理学等领域的基础上，是多学科发展演变而来，如今成为智能控制的重要组成部分。目前人类可借助强化学习解决人工智能领域的问题。强化学习与深度学习不同的是，它是使用心理学和神经学的思想来解决问题。深度学习的思想是通过神经网络将底层数据转变成机器易于识别的高级信息，更专注于对事物特征的描述。而机械臂使用强化学习算法训练时，需要输入高维状态信息，将历史经验迁移到新环境中进行学习。与深度学习不同的是，强化学习的思想是使用策略来达成目标，在不断试错的过程中累计最大奖励，并学习到最佳策略。随着技术进步，深度学习和强化学习运用在生活中的需求更大。

在 1954 年，明斯基首次提出了概念，接着两年之后，贝尔曼在明斯基的基础上提出了著名的马尔科夫决策的方法。强化学习在此之后得到了重大发展，并都以贝尔曼的思想为基础。沃特金斯在 1989 年提出了常用的算法 Q-Learning 和 Sarsa 和蒙特卡洛方法。而美国的 DeepMind 公司在 2013 年首次提出深度 Q 网络(Deep Q Network，DQN)，在 Q 学习的基础上引入了深度神经网络，从而实现了对高维状态信息的处理。紧接着 Wang 等人[10]提出竞争网络(Dueling Network)，使用神经网络去拟合状态价值和优势值，该框架不仅提高了最终的效果，还能和优先级经验回放结合在一起。DQN 使用卷积神经网络处理高纬度的图像信息，广泛用于游戏领域。该算法与 DQN 相比，最大的不同点是将状态和动作分开，在运算过程中，状态不会过度依赖状态的价值，可以使用状态单独预测。

策略梯度方法最早是在 21 世纪初提出来的，接着置信区间策略优化（Trust Region Policy Optimization, TRPO）提出[11]。在这之后，Deepmind 在 TRPO 的基础上提出邻近策略优化算法（Proximal Policy Optimization, PPO）的方法，目前 PPO 成为主流算法，GPT 智能搜索引擎则是使用 PPO 实现部分功能的。Silver 等人[12]提出确定性策略梯度（DPG），策略函数直接输出动作值，而不是根据概率选择。Scott 等人[13]提出双延迟深度确定性策略梯度(Twin Delayed Deep Deterministic Policy Gradient,TD3)强化学习算法，减少了的价值过度估计的问题。例如目前前景巨大的新能源汽车行业，图像识别、智能感知和导航成为研究热点，Wang 等人[14]首次将 DDPG 算法应用在智能驾驶上，用于解决来连续控制问题中复杂状态的

问题，深度强化学习的重要性凸显出来。Mnih[15]等人提出了一种值函数与策略函数结合的方法，异步动作-评估法（Asynchronous Advantage Actor-Critic,A3C）。最后 Haarnoja 等人在 Mnih 的基础上提出了 SAC 算法。

Daniel Udekwe[16]等人针对控制器参数的优化问题，提出一种基于深度强化学习的 PID 控制器，开发了多个强化学习算法平台，包括 SAC、DDPG、TD3 算法。使得它们能够在一个无限搜索空间内调节 PID 控制参数，从而实现了自适应调整，精准得轨迹跟踪和实现系统的稳定。将该方法和基准方法进行对比取得了较好的效果。金堃[17]等人使用分布式的 PPO 算法和简单的前向传播奖励函数，在多种难度较大的地形下，成功完成对多个虚拟人物的训练。而 OpenAI 提出对 PPO 算法进行改进，已经成功训练出多腿机器人相互玩游戏，并指导机器人适应对方策略的变化。加州大学提出的策略搜索算法，通过该算法训练拟合线性模型来优化动作轨迹[18]。上述研究学者实现对强化学习算法理论的研究，同时将强化学习算法应用于现实中，例如智能驾驶、机器人等。为之后深度强化学习算法更深层次的应用打下基础。

1.2.3 深度强化学习奖励机制的研究现状

在深度强化学习中，智能体通过接收环境中的状态并反馈出一种信号，称之为奖励。智能体则通过强化学习算法，学会在一个或者多个回合得到最大奖励值，并通过不断训练，实现长期奖励的最大化。与奖励值最直接的影响因素是强化学习的奖励函数设计。奖励函数的优劣对训练效果起到决定性作用，是技术人员根据任务需求设计得到。机械臂的运动规划通常使用稀疏奖励或稠密奖励。稀疏奖励也称为二值化奖励，当使用稀疏奖励时，机械臂探索动作空间得到的奖励，大多数时候会是无效奖励，因此样本利用率较低[19]。

赵志龙[20] 针对基于深度强化学习的机械臂轨迹规划方法学习效率较低，规划策略鲁棒性差的问题，提出了一种基于语音奖励函数的机械臂轨迹规划方法，利用语音定义规划任务的不同状态，并采用马尔科夫链对状态进行建模，为轨迹规划提供全局指导，降低深度强化学习优化的盲目性。李勇[21]等人为了提高机械臂在狭窄通道的情况下的避障能力，提出一种使用 APF 算法对奖励函数进行设计，再利用 DDPG 算法进行训练，大大提高了收敛效率。付子强[22]等人为了解决 DDPG 算法应用于机械臂运动规划时样本利用率低的问题，提出多经验池延迟采样机制，并设计了位置奖励函数，有效提高了学习效率。ZHENG Z[23]等人针对机械臂手眼标定的问题，提出一种神经动力学自适应奖励函数和动作函数相结合的方法，并在实验证明了其有效性。罗针对机械臂操作任务采用固定奖励函数时，密集奖励函数影响性能，采用网络重置和将经验转化为稀疏奖励，大大提升了性能。上述

学者针对机械臂实际应用中的问题，通过使用传统算法以及对样本采样的研究等，或者采用多种奖励函数结合的方法，实现对强化学习算法的改进，在实际应用中也取得了不错的效果。

为了高效得利用数据，最近发表的基于经验优先级回放和事后经验回放可以取得很好的效果。但面对不确定性环境，单纯的稀疏奖励无法感知无障碍物之间的关系。同时 Agent 到障碍物和目标点之间的实时距离决定了任务完成的质量和效果。对应的 Bhuiyan 等人[24]和 Chen L 等人[25]采用机械臂到障碍物的最短距离和机械臂末端到目标点的欧式距离作为奖励函数。考虑到避障跟到达同时进行时，欧氏距离会鼓励机械臂采取最短路径接近目标，机械臂则会陷入局部最优解。

1.2.4 深度强化学习在机械臂控制中的研究

基于深度强化学习的机械臂研究，最主要的目标是如何通过增大深度强化学习的能力，使得机械臂发挥最高效率。在充满不确定性的复杂环境中，利用深度学习对环境的感知能力对视觉图像或者传感器输入做特征提取，作为强化学习算法的输入。机械臂在这种自我感知能力下，通过不断试错学习策略，积累奖励值的最大化，从而学习到最佳策略。机械臂自主轨迹规划的研究，目前在运动控制、视觉抓取等方面取得较好的效果。下面论述了最新的研究进展。

Fei Wang[26]等人针对孔洞装配的任务中依赖使用演示信息会降低收敛速度的问题，提出了一种改进的深度学习算法，利用近端策略优化的演示信息机制和 LSTM，为机器人建立先验知识库并建立演示信息的使用机制，提高了数据工作效率。实验结果表明，LSTM 网络的引入可以提高了装配任务特征的强化学习收敛性，增强了机械臂轨迹的稳定性。

刘勇等[27]人在研究机械臂追踪和避障的运动控制方法时，将 DDPG 算法和 PID 控制技术融合，使用 PID 技术控制机械臂快速到达工作平面内，实现接近目标并重合在一起。再使用 DDPG 算法训练机械臂在工作区域学习追踪目标物体在平面内的映射同时避开障碍物，实现机械臂的追踪与避障任务。与传统 DDPG 算法相比，引入 PID 技术的改进算法获得较高的奖励值，对比有较好的收敛效果和控制性能。

李[28]针对机械臂在实际调试时间长、稳定性差的问题，提出一种近端策略优化算法（PPO），通过状态变量实现仿真环境与深度强化学习算法的交互，通过奖励函数实现对神经网络参数的训练，能够实现快速收敛并且有效降低抖动现象。李[29]在强化学习基础上提出针对机械臂控制模型的先 2D 后 3D 的训练方法，在取得应用效果的情况下，相比于直接在 3D 模型中的训练，缩短了近 52%的平均训练时长。江达等人[29]针对连续型机械臂系统追踪问题，提出基于强化学习的数据

驱动预测模型，采用 BP 神经网络对三臂节连续型机械臂进行建模，并作为预测模型指导强化学习实时调节所提出滑模控制器的控制参数，从而实现连续型机器人运动的实时跟踪控制，可将误差控制在±1%内。

Andy Zeng[30]等人针对在复杂环境中机械臂的选择性抓取问题，采用深度强化学习方法实现机械臂对杂乱物体的协同抓取。首先使用两个训练国的卷积神经网络，两个网络都是基于深度 Q 学习的网络，一个负责推算末端夹爪的夹取方向和对物体像素进行采样，另外一个网络负责分析和执行末端夹爪的动作。实现在试错中学习最佳策略。抓取到物体也会有奖励值的实时反馈。通过马尔可夫决策的方法进行学习抓取动作，同时结合过去的状态信息，更好实现抓取动作。经过实验证明，机械臂在学习期间，在复杂物体堆中可以完成抓取任务，达到很好的抓取效果。

Zhu Z[31]等人提出一种将深度强化学习与迁移学习相结合的机械臂运动规划新框架，首先在无障碍物环境中训练机械臂，再转移到有障碍物的环境中进行训练。将训练过的 DDPG 策略通过迁移学习转移到新环境中。经过实验验证，机械臂在规划抓取姿态时，相对于传统 DDPG 算法，可以有效避开障碍物，并更快完成抓取姿态。Binzhao Xu[32]等人针对目前研究较少的机械臂动态抓取的问题，设计出基于强化学习的动态抓取模块和基于 LSTM 的轨迹预测模块，将动态抓取分为基于强化学习的抓取策略学习和轨迹预测模块。Binzhao Xu 首先在静态障碍物的模拟环境中进行训练，再迁移到现实实验中训练。结果表明基于 LSTM 的轨迹预测结果有所增强，作者通过设置机械臂操纵器和移动汽车上放置障碍物，实验表明该方法在不同速度下机械臂都能抓取到不同轨迹上的物体。

Kwan-Woo Park[33]等人针对工作空间中存在固定和移动障碍物时，机械臂的避障运动规划问题，考虑到高维输入和连续动作空间的问题特性，采用深度强化学习算法 SAC 进行训练，奖励函数采用稀疏奖励，同时用 LSTM 结构作为预测模块来预测障碍物未来的位置，并将预测位置加入到新的轨迹中，实验表明该改进算法可以取得不错的避障效果。

综上所述，深度强化学习算法在机器人应用中有很多进展和成果，在机械臂轨迹规划方面也有较为成熟的技术。这些算法能够使得机器人应用于实际环境中，同时考虑避障和任务目标还需继续提升技术。RL 算法依然面临各种挑战，如算法易导致过度探索边界值和收敛速度慢。单纯改进算法来获得最佳运动规划效果是不够的。尤其是在面临不确定性环境时，数据质量的优化和奖励函数的设计，对于运动规划显得很重要。

1.3 主要研究内容和章节安排

1.3.1 主要内容

本文以复杂环境下的机械臂自主轨迹规划为研究背景，分析了当前国内外机械臂的路径、轨迹规划的算法和深度强化学习算法在机械臂控制的研究现状。针对传统机械臂轨迹规划存在对环境的依赖度较高，无法适应非结构化环境的问题，以软演员-评论家（Soft Actor-Critic，SAC）算法为基线，将它应用于动态环境中，自主完成轨迹规划任务。针对深度强化学习中存在的机械臂的问题进行改进，改进了奖励函数和 SAC 算法（AL-PER-SAC）。在 PyBullet 环境中搭建仿真实验环境，使用基于 LSTM 的 SAC 算法（LSTM-SAC）、融合注意力机制和 LSTM 的 SAC 算法（AL-SAC）和 AL-PER-SAC 算法进行对比试验，研究内容如下：

（1）研究机械臂的运动学，根据机械臂 D-H 参数，使用 MATLAB 建立机械臂模型，求解出工作空间并以此为根据设计强化学习模型，在仿真环境中搭建实验平台，设计状态空间、动作空间和奖励函数。

（2）针对现有的强化学习奖励函数进行改进，并使用主流算法近端策略优化（PPO）和 SAC 算法开展实验，验证了改进奖励函数的可行性。

（3）针对不确定性环境（动态环境）的特点，提出了基于 SAC 算法的改进算法 AL-PER-SAC。改进点如下所示：

1.针对有相关研究人员提出利用长短期记忆神经网络 LSTM 预测动态障碍物技术为基础。本文针对复杂环境的特点，提出融合注意力机制和 LSTM 的 SAC 算法，利用注意力机制对复杂环境中的状态进行重要性分配，再使用 LSTM 让机械臂做出更合理的决策。

2.结合优先经验回放机制，加快算法采样的效率，提高收敛速度。同时利用高价值样本，让机械臂更好得处理环境中的信息。

1.3.2 章节安排

本文分为五章，具体安排如下：

第一章为绪论。对本课题的研究背景与意义进行介绍。通过查阅大量国内外关于机械臂运动规划算法以及深度强化学习在机械臂运动规划的研究现状，对多自由度机械臂运动规划算法进行分析和总结。

第二章为深度强化学习理论部分，首先分析了机器人的运动学坐标转换原理。然后介绍了马尔科夫决策的过程，以及各个部分的组成。然后介绍了基于值函数、策略和演员-评论家的深度强化学习算法，为之后使用深度强化学习算法进行机械臂的自我轨迹规划的研究奠定了理论基础。

第三章为机械臂建模和环境搭建，首先介绍了机器人的姿态描述，并就本文使用的 Franka Panda 机械臂总结 D-H 参数，分析机械臂的运动学。搭建本文的实验环境，包括定义了强化学习算法的状态空间、动作空间、奖励函数。针对现有的机械臂运动规划问题的奖励函数进行改进，引入实时指导奖励机制和二值化奖励，并使用主流算法 PPO 和 SAC 算法，以机械臂的平均成功率、安全性和奖励值作为评价指标并开展两个实验，验证了改进奖励函数的优越性。

第四章主要针对 SAC 算法的改进。首先根据仿真环境的特殊性，使用融合 LSTM 网络的注意力机制对 SAC 算法进行改进。再结合优先级经验回放机制，提出了 AL-PER-SAC 算法。通过仿真实验验证了改进算法的优越性。

第五章为本文的总结与展望，总结了本文的主要研究内容，并对其中的不足和未来的展望。其中总结了本人在学习期间做的一些准备工作，为以后条件成熟时，将强化学习算法部署到真实机器人做准备。其中包括将树莓派 4B 作为 PC 端，利用 ROS 机器人系统来实时控制真实机械臂，在机械臂上部署深度摄像机。为以后实现本文的算法研究做铺垫。

第 2 章 相关理论概述

2.1 引言

前一章介绍了机械臂轨迹规划的国内外研究进展，包括分析了传统轨迹规划方法以及限制条件。引出了针对复杂的不确定性环境下，深度强化学习方法是钟实现的可能。本章内容就深度强化学习展开，主要介绍了深度强化学习的发展史和相关算法原理介绍，为之后的算法引入、算法改进打下基础。

2.2 机械臂位姿变换

2.2.1 机械臂的姿态描述

机械臂是可以代替人类手臂的机械设备，本文使用的 Franka Emika 机械臂由底座和七个连杆组成，其中连杆与连杆之间通过转动关节进行连接。Franka Emika 机械臂是冗余机械臂，由于人类手臂也是七自由度，从仿生学看，该机械臂更符合人体工程学。在实现本体避障避、开关节限位、增加操作度等方面有很大优势。如图 2-1 为运动学模型简化图。机械臂的底座一般作为世界坐标，即为机械臂在空间坐标系中的原点。

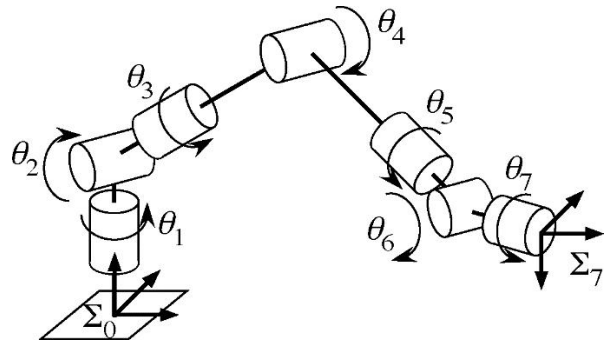


图 2-1 七自由度机械臂简化图

Fig2-1. Simplified diagram of a seven-degree-of-freedom robotic arm

机械臂的位姿表示，即为在工作空间中，机械臂底座和各个转动关节和末端执行器位置与姿态的描述关系。在对机械臂运动学进行研究时，需要对机械臂本体建立空间坐标系，描述清楚机械臂的空间位姿。首先对底座建立世界坐标，然后再通过一系列计算获得各个转动关节和末端执行器的位置和相互关联。

在对机械臂进行位姿说明时需要建立三维坐标系{b}，一个刚体m在空间坐标系中的表示方式为$m_a = (m_x\ m_y\ m_z)^T$，其中$m_x\ m_y\ m_z$为在三个坐标轴垂直方向的投影。同样，如图（2-2）所示，坐标系也是可以叠加的，这样可以表示在机械臂的坐标空间下，一个点到另一个点的空间位置关系

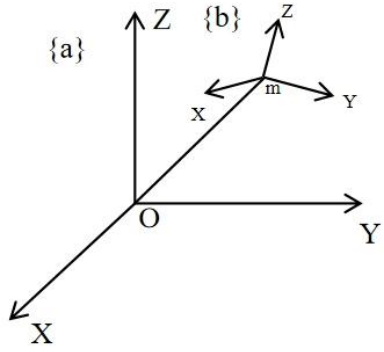


图 2-2 空间中m点的位置描述

Fig2-2. Description of the position of an m-point in space

同样在坐标系{a}下，m点的表示方式为$m_a = (X_b^a \ \ Y_b^a \ Z_b^a)^T$，点 m 表示为坐标系与参考坐标系的夹角的余弦所组成的矩阵表示，即为旋转矩阵${}_a^bR$，如式子(2-1)所示：

$$ {}_a^bR = \begin{bmatrix} \cos(X_a, X_b) & \cos(X_a, X_b) & \cos(X_a, X_b) \\ \cos(Y_a, X_b) & \cos(Y_a, X_b) & \cos(Y_a, X_b) \\ \cos(Z_a, X_b) & \cos(Z_a, X_b) & \cos(Z_a, X_b) \end{bmatrix} \tag{2-1} $$

2.2.2 机械臂坐标系变换

上文已经对机械臂的位姿表示作了详细说明。以基座标作为参考坐标系，机械臂的关节点或者末端执行器在基座标系下进行位姿变幻时，可以通过计算得到它们之间的位姿关系。

平移变换。如图 2-3 所示，坐标系{b}是在基座标系{a}下进行平移操作后的位置，由此可以实现两个坐标系之间的平移变换。其中m_b表示为 m 点在坐标系{b}下的位置向量，m_a^b表示目标坐标系{b}在基座标系{a}下的位置向量。由此可以看出坐标系{b}下的点 m 相对于基座标{a}的位置向量可以表示为：

$$ m_a = m_a^b + m_b $$

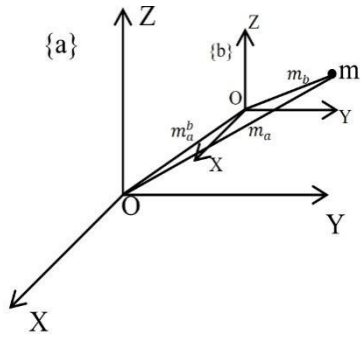


图 2-3 平移变换

Fig2-3. Translational change

旋转变换。如图 2-4 所示，坐标$\{b\}$跟参考坐标系$\{a\}$有相同的原点，坐标系{b}是在基座标系$\{a\}$下进行旋转操作后的位置，由此可以实现两个坐标系之间的旋转

变换。其中m_b表示为 m 点在坐标系$\{b\}$下的位置向量，${}_a^bR$表示目标坐标系$\{b\}$在基座标系$\{a\}$下的旋转矩阵。由此可以看出坐标系$\{b\}$下的点 m 相对于基座标$\{a\}$的位置向量可以表示为：

$$m_a = {}_a^bR \times m_b$$

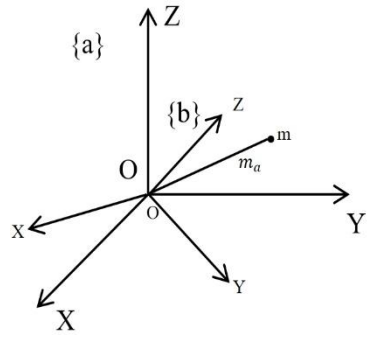


图 2-4 旋转变换

Fig2-4. Rotary transformation

复合变换。如图 2-5 所示，坐标系{b}是在基座标系{a}下进行平移和旋转操作后的位置，由此可以实现两个坐标系之间的平移和旋转变换。首先坐标系{b}是在基座标系{a}下进行平移操作，其中m_b表示为 m 点在坐标系{b}下的位置向量，m_a^b表示目标坐标系{b}在基座标系{a}下的位置向量。${}_a^bR$表示目标坐标系{b}在基座标系{a}下的旋转矩阵，由此可以看出坐标系{b}下的点 m 相对于基座标{a}的位置向量可以表示为：

$$m_a = {}_a^bRm_a^b + m_b$$

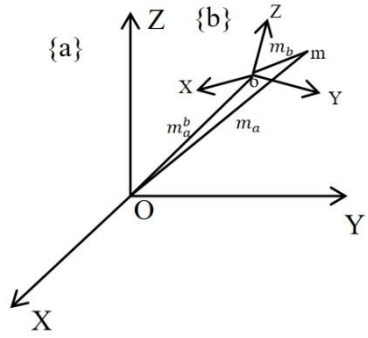


图 2-5 复合变换

Fig2-5. Complex transformation

机械臂的姿态变换是通过齐次变换法实现的，其中位姿代表着机械臂的位置和姿态。通过这种方法将机械臂的运动、矩阵变换等联系在一起。在执行任务时，在关节空间或者笛卡尔空间下，通过控制关节角度或机械臂的执行器位姿，实现机械臂的运动规划。智能化时代下发展迅速的强化学习方法可以为机械臂的位姿变换以及运动规划提供了一个智能化的架构。

2.3 马尔可夫决策

在强化学习的早期阶段，人们分别对试错学习、时序差分和最优控制等方向进行了深入研究。在最优控制方向上，提出了马尔可夫决策过程。最优控制理论的主要作用是充当动态系统中的控制器。针对这一理论，贝尔曼[34]提出了动态规划的求解方法，通过这种方法引出了马尔可夫决策过程（Markov Decision Process,MDP）理论。20 世纪 60 年代，Howard 基于马尔可夫决策过程研究了值迭代和策略迭代等方法，逐渐奠定了现代强化学习的理论基础。

马尔可夫决策过程由智能体与环境之间的相互作用组成，包含四个关键要素：状态、行动、策略和奖励。这里的状态指的是强化学习环境中除智能体之外的其他元素，在机械臂轨迹规划问题中，机械臂的状态是指机械臂末端的位置、速度和角速度，以及目标点的位置和速度。对于移动机器人使用强化学习进行导航研究时，智能体的动作是离散的状态，因此状态空间也是离散的状态。而机械臂轨迹规划问题中，机械臂的动作是连续的，状态空间可以通过深度学习来拟合。

动作是指智能体根据环境提供的状态信息，经过交互得到的执行方法。机械臂的动作在欧拉坐标系下的表达形式为各个转动关节的旋转角，可以表达为$(\theta_1, \theta_2, \theta_3)$。在笛卡尔坐标系下可以表示为末端执行器的坐标位置$(x, y, z)$。

策略是指智能体根据环境提供的状态信息，根据策略来选择相应的动作。对于离散动作来说，策略网络首先会根据状态信息选择离散动作中，概率较大的动作，然后采取相对应的策略。而对于连续动作来说，策略会根据状态信息直接计算出动作。

奖励则是智能体在执行完动作后，根据智能体与环境任务之间的关系，环境根据奖惩函数反馈出来的奖励值。奖励函数的优劣对算法训练的效果有直接的影响。马尔可夫决策过程是指在t时刻的状态s_t，只和t-1时刻的状态s_{t-1}以及动作a_{t-1}有关，与历史无关，马尔可夫决策图如图 2-6 所示。

在整个过程中，智能体所处的初始状态是s_0，然后根据策略$\pi(a|s)$采样出动作a_0，$a_0 \sim \pi(a_0|s_0)$,然后根据状态转移概率$p(s'|s,a)$采样新的状态s_1，这样可获得单步奖励r_1。然后重复这样的行为，一直到最终状态s_3，可以得到一条轨迹$\tau = (s_0, a_0, s_1 \cdots, s_t)$。定义 以上的转移路径后，得到的回报函数之和如下（2-2）所示：

$$R(s_0, a_0) + \gamma R(s_1, a_1) + \gamma^2 R(s_2, a_2) + \cdots \quad (2\text{-}2)$$

若奖励值只和状态有关，那么上式可写成公式(2-3)：

$$R(s_0) + \gamma R(s_1) + \gamma^2 R(s_2) + \cdots \quad (2\text{-}3)$$

马尔科夫决策的最终目标是选择一组最佳的动作，使得全部的奖励值加权期望最大，如下公式（2-4）所示：

$$maxE[R(s_0)+\gamma R(s_1)+\gamma^2 R(s_2)+\cdots] \quad (2\text{-}4)$$

从上式可以发现，在 t 时刻的奖励值打了γ^t的折扣，可以看出这是个衰减的过程，越往后的状态对奖励值和的影响越小，最大化期望也将最大的奖励值$R(s_i)$放在前面，越小的放在后面。

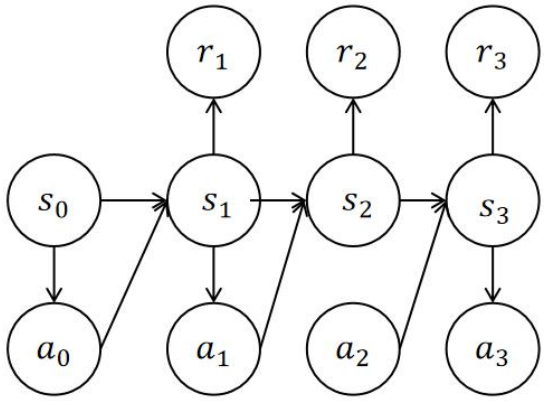


图 2-6 马尔可夫决策过程

Fig2-6. Markov decision process (mathematics)

2.4 深度强化学习算法

2.4.1 基于值函数的强化学习算法

经典的强化学习 Q-learning 算法与神经网络相结合，形成了深度 Q 网络（Deep Q network，DQN）算法，有效地解决了 Q 学习计算效率低和数据内存有限的问题。DQN 算法的网络结构由目标网络和估计网络组成；DQN 算法一般通过贪婪策略的最大化原则来选择 Q 值，这会导致算法估计过高，产生较大偏差。

然而，DQN 存在高估策略值的问题，为解决这一问题，VanHasselt 等人[34] 提出了双 DQN（DDQN）算法，将行动值计算和行动选择分开。Wang 等人[35]提出的决斗 DQN 算法优化了 DQN 算法的结构。该算法将网络分为两部分：价值函数和支配函数。Dueling DQN 算法将价值函数和支配函数的线性组合作为最终输出，比 DQN 算法更具表现力。

算法的损失函数如下公式(2-5)所示：

$$L=\left(r+\gamma maxQ\left(s^{'},a^{'},\omega\right)-\widehat{Q}(s,a,w)\right)^2 \quad (2\text{-}5)$$

2.4.2 基于演员—评论家的深度强化学习算法

演员-批评家（Actor-Critic，AC）算法的原理是，演员部分通过探索环境生成行动集合，并根据概率函数分布选择行动。评论家部分对演员的行为进行评估。

演员根据评论家的评分优化动作概率函数，最后指导智能体选择最优动作。

针对Critic网络的更新方式，采用均方根误差损失函数来更新网络，如下公式(2-6)和(2-7)所示：

$$\delta_t = r_{t+1} + \gamma V(s_{t+1}) - V(s_t) \tag{2-6}$$

$$\omega_{t+1} = \omega_t + \alpha_\omega \delta_t \nabla Q(s_t, a_t) \tag{2-7}$$

式中，δ_t为TD误差，α_ω为策略网络参数的更新频率。其中TD是指一种基于时间误差的学习方法，通常用于值函数的估计和策略的优化。TD误差指模型的预测值与实际值的误差，用于衡量强化学习模型训练的准确性。

异步优势演员-评论家（Asynchronous Advantage Actor-Critic，A3C）算法相对于 AC 算法的改进包括训练框架异步化、网络结构优化以及评估点优化等 [36]。A3C 算法使用了 DQN 算法的经验回放，并利用多线程方法允许多个智能体同时与环境交互，将每个智能体的训练数据汇集到一个公共空间，以便每个智能体都能从公共空间中采样数据并共享，并指导自身策略学习，从而解决交流算法难以收敛的问题。

2.4.3 基于策略的深度强化学习算法

在确定性策略梯度（Deterministic Policy Gradient，DPG）算法的基础上，Lillicrap 等人[37]提出了深度确定性策略梯度（DDPG）算法，该算法融合了 DQN 和 AC 算法的思想。DDPG 的网络结构分为一个动作网络和价值网络，其中包含四个神经网络。评论家模块利用时差误差（TD-error）更新网络参数ω，并定期复制ω到目标网络。演员模块通过算法更新网络参数，行动策略根据其策略网络的输出结果选择动作作用于环境[38]。DPG 的公式如下公式(2-8)所示：

$$\nabla_\theta L(\mu_\theta) = E_{s\sim\rho^\mu}\left[\nabla_\theta \mu_\theta(s) \nabla_a Q^\mu(s,a)|_{a=\mu_\theta(s)}\right] \tag{2-8}$$

DDPG 算法比 DQN 和 AC 算法[39]更稳定，可以处理连续的动作空间任务。不过，DDPG 算法对超参数的变化很敏感，需要长期微调参数才能获得更好的算法性能。此外，评论家的 Q 函数存在高估 Q 值的问题，这可能会导致动作策略学习不足，并收敛到非最佳状态。双延迟 DDPG 算法（Twin Delayed DDPG，TD3）[40]对 DDPG 算法的优化包括三个部分：采用双 Q 网络来解决高估评论家 Q 函数的 Q 值估值问题；通过延迟动作网络的策略更新，使得网络的训练更稳定。利用目标策略平滑法，在计算动作目标网络 Q 值的过程中加入噪声，使网络准确计算行动目标网络的 Q 值。通过使用目标策略平滑方法，在计算动作目标网络 Q 值的过程中添加噪声，从而使网络准确且鲁棒性高。

SAC 算法的发展是在 TD3（Twin Delayed DDPG）和 DDPG 算法基础上的[41]。

由于 DDPG 算法是一种用于连续动作空间的深度强化学习算法，应用于机械臂的运动规划较为广泛，但由于 DDPG 算法在训练时易遇到边界值，导致训练效果不稳定。而 TD3 则是在 DDPG 的基础上引入了双网络和延迟更新，进一步提升系统的性能。SAC 算法是一种基于值函数和策略的算法，是在 TD3 的基础上进一步发展，引入熵正则化[42]，旨在最大化未来积累的奖励值和熵，使策略尽可能随机，并避免机械臂重复探索运动，使得它可以更好得用于高维机械臂的轨迹规划问题[43]。

SAC 包含一个策略网络$\pi_\theta(a_t, s_t)$，两个软 Q 网络$Q_{\theta 1,2}(a_t, s_t)$、行为价值网络$V_\varphi(s_t)$和目标价值网络$V_\varphi^-(s_t)$。目标函数定义如公式(2-9)所示：

$$J(\pi) = \sum_0^T E_{(s_t,a_t)\sim\rho^\pi}\left[r_{(s_t,a_t)} + \alpha H(\pi\cdot|s_t)\right] \tag{2-9}$$

其中，α表示熵正则化系数，用于控制最优策略的随机性程度，并衡量熵相对于奖励的重要性。表示策略下的分布。

SAC 的 Q 值可以用基于熵值改进的贝尔曼方差[44]来计算，价值函数定义如(2-10)所示：

$$Q(s_t, a_t) = E_{a_t\sim D}[r(s_t, a_t) + \gamma V^\pi(s_{t+1})] \tag{2-10}$$

其中s_{t+1}是从经验池 D 中采样得到。而策略网络则是通过最小化 Kullback-Leibler(KL) 散度[45]来更新如(2-11)所示：

$$J_\pi(\emptyset) = E_{a_t\sim\pi, s_t\sim D}\left[log\pi_\emptyset(s_t, a_t) - \min_{i=1,2} Q_{\theta_i}(s_t, a_t)\right] \tag{2-11}$$

PPO 算法是一种基于策略的强化学习算法，它的出现是为了解决策略梯度下降算法在处理连续动作时学习率难以确定的问题。PPO 算法使用重要性采样来修正策略更新时的分布差异[46]。重要性采样通过计算动作在新旧策略下的概率比值，来调整策略梯度的方向和幅度。这样可以更准确地估计策略更新的效果，并提高算法的收敛性。重要性采样是 PPO 算法中的一种近似采样方法，它可以提高算法的效率，并将其从在线算法转变为离线算法。重要性采样的公式如下(2-12)：

$$E_{x\sim p}[f(x)] = E_{x\sim q}\left[\frac{\rho(x)}{q(x)} f(x)\right] \tag{2-12}$$

PPO 算法具备了策略梯度下降算法的部分优点，即采样数据和使用随机梯度上升方法优化代替目标函数之间交替进行并提出新的目标函数，实现小批量的更新。公式如下(2-13)所示：

$$J^{\theta'} = E_{(s_t,a_t)}\sim\pi_{\theta'}\left[\frac{p_\theta(a_t|s_t)}{p_{\theta'}(a_t|s_t)} A^{\theta'}(s_t, a_t)\right] \tag{2-13}$$

PPO 中的优势估计[47]，就是期望回报实际上的 Q 值，每次 rollout 就是对 Q 的一次采样可以定义为优势 A，即为 Q 和 V 的差值。由于目前方差比较大，选择引入 bootstrap 的方法减小方差，如 TD 误差，对 Advantage 的一次采样为公式(2-14):

$$advantage = r + \gamma v(s_{t+1}) - v(s_t) \quad (2\text{-}14)$$

2.5 本章小结

本章首先介绍了机械臂运动学中坐标变换的原理，为下一章机械臂运动学的分析做了铺垫。 接着，介绍并分析了强化学习的发展历史和强化学习算法的基本原理，为第四章通过改进 SAC 算法来研究机械臂轨迹规划奠定了理论基础。

第3章 基于深度强化学习的机械臂轨迹规划算法建模及改进

3.1 引言

传统机械臂轨迹规划难以适应环境的变化，在面对动态变化时，难以做出合理的判断。因此本章针对以上问题，首先分析机械臂的运动学模型，并根据复杂环境的需求建立 PyBullet 仿真环境下的机械臂动态环境，设计了强化学习的状态空间、动作空间和原始奖励函数。并针对单一稠密奖励函数收敛慢、成功率低等问题，提出了融合实时指导奖励和二值化奖励的改进奖励函数，并在两个实验环境下验证了改进奖励函数的效果。

3.2 机械臂运动学分析

本节将介绍机械臂的运动学理论，包括机械臂的正向运动学和逆向运动学。机械臂建模时通常会选择合适的位置建立笛卡尔坐标系，当确定了世界坐标（底座的坐标系）时，通过机械臂各连杆和末端执行器的笛卡尔坐标之间的相对关系来确定机械臂各连杆和末端位姿[48]。正向运动学是指当机械臂相关转动关节满足一定角度或者一定的旋转速度时，机械臂可以通过旋转矩阵得到末端执行器在空间中的对应位姿。

在机械臂技术中，由雅克·德纳维特和哈滕伯格提出的 Denavit-Hartenberg 参数，也称为 D-H 参数，是常用的选择参考框架的惯例，用于将参考坐标系结合到机械臂的连杆上[49]。D-H 参数如下元组(3-1)所示。D-H 法的建模基本原理是通过四个参数，建立相邻两个连杆的位置关系，对于描述连杆 i-1 的坐标系{i-1}建立在关节 i 的轴线上，相应变换矩阵所需要的 D-H 参数为：

$$ {}^{i-1}_{i}T: \{a_i, \alpha_i, d_i, \theta_i\} \tag{3-1} $$

其中，四个参数分别是机械臂的连杆长度b_i，连杆转角α_i，连杆偏置d_i，两连杆关节角θ_i。建立简易坐标系如图 3-1 所示。

首先需要确定关节坐标系的 X 轴和 Z 轴方向。Z 轴方向与机械臂关节轴线重合，可通过右手法则来确定。基本的运动可以描述为按顺序依次绕 Z 轴旋转θ、沿着 Z 轴平行 d、沿着 X 轴平行 a、绕 X 轴旋转α。坐标系的原点是两相邻不相交轴线的公垂线与轴线的交点，分别为图 3-1 中的b_{i-1}和b_i与关节(i-1)和关节(i)的交点。X 轴方向可表示为如下三种情况：

(1)当两轴线不相交时，X 轴与公垂线重合，方向为从 i-1 到 i。

(2)如果两轴线相交时，X 轴为两相邻轴线所构成的法线。

(3)若两轴线重合，X 轴与轴线垂直并且使其他连杆参数 都设置为 0。

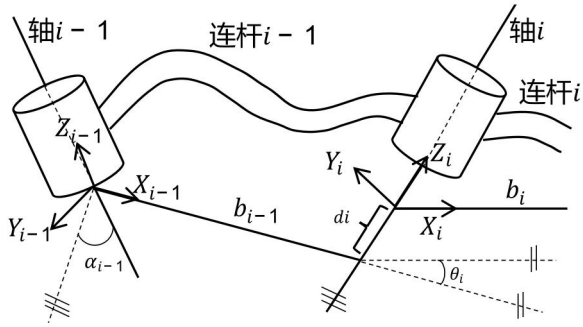


图 3-1 机械臂连杆关系图

Fig3-1. Robotic arm linkage relationship diagram

上述所说明的变换可用公式(3-2)和(3-3)表示如下：

$$T=\begin{bmatrix} cos\theta_{n+1} & -sin\theta_{n+1} & 0 & 0 \\ sin\theta_{n+1} & cos\theta_{n+1} & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & d_{n+1} \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$\begin{bmatrix} 1 & 0 & 0 & a_{n+1} \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}=\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & cos\alpha_{n+1} & sin\alpha_{n+1} & 0 \\ 0 & sin\alpha_{n+1} & sin\alpha_{n+1} & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (3\text{-}2)$$

$$_{n+1}^{n}T=\begin{bmatrix} cos\theta_{n+1} & -sin\theta_{n+1}cos\alpha_{n+1} & sin\theta_{n+1}sin\alpha_{n+1} & a_{n+1}cos\theta_{n+1} \\ sin\theta_{n+1} & cos\theta_{n+1}cos\alpha_{n+1} & -cos\theta_{n+1}sin\alpha_{n+1} & a_{n+1}sin\theta_{n+1} \\ 0 & sin\alpha_{n+1} & cos\alpha_{n+1} & d_{n+1} \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (3\text{-}3)$$

在本文中使用的为 Franka Emika 机械臂，该机械臂末端位姿有 7 个自由度，可以实现末端的任意位姿。这是一款 7 自由度机械臂，按照人类手臂的结构设计而来[50]。包括如图 3-2 所示的 7 个转动关节，

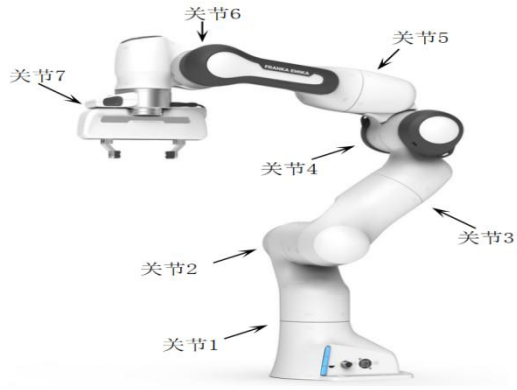


图 3-2 机械臂连杆关系图

Fig3-2. Robotic arm linkage relationship diagram

下面就以 Franka Emika 机械臂为例，其 D-H 参数列表如下表 3-1 所示：

表 3-1 Franka Emika 机械臂 D-H 参数[51]

Tab.3-1 Franka Emika Robotic Arm D-H Parameters

连杆 i	a/mm	d/mm	α/deg	θ/deg	范围/rad
1	0	333	-90	θ_1	(-2.897,2.897)
2	0	0	90	θ_2	(-1.763,1.763)
3	82.5	316	90	θ_3	(-2.897,2.897)
4	-82.5	0	-90	θ_4	(-3.07,-0.07)
5	0	384	90	θ_5	(-2.897,2.897)
6	88	0	90	θ_6	(-0.02,3.752)
7	0	107	0	θ_7	(-2.897,2.897)

根据上表 3-1 的 D-H 参数和计算得到的齐次变换公式，可以获得各个连杆的变换矩阵：

$$
{}^{0}_{1}T=\begin{bmatrix} cos\theta_1 & -sin\theta_1 cos\alpha_1 & sin\theta_1 sin\alpha_1 & a_1 cos\theta_1 \\ sin\theta_1 & cos\theta_1 cos\alpha_1 & -cos\theta_1 sin\alpha_1 & a_1 sin\theta_1 \\ 0 & sin\alpha_1 & cos\alpha_1 & d_1 \\ 0 & 0 & 0 & 1 \end{bmatrix}=\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & -1 & 0 \\ 0 & -1 & 0 & 333 \\ 0 & 0 & 0 & 1 \end{bmatrix}
$$

$$
{}^{1}_{2}T=\begin{bmatrix} cos\theta_2 & -sin\theta_2 cos\alpha_2 & sin\theta_2 sin\alpha_2 & a_2 cos\theta_2 \\ sin\theta_2 & cos\theta_2 cos\alpha_2 & -cos\theta_2 sin\alpha_2 & a_2 sin\theta_2 \\ 0 & sin\alpha_2 & cos\alpha_2 & d_2 \\ 0 & 0 & 0 & 1 \end{bmatrix}=\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & -1 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}
$$

$$
{}^{2}_{3}T=\begin{bmatrix} cos\theta_3 & -sin\theta_3 cos\alpha_3 & sin\theta_3 sin\alpha_3 & a_3 cos\theta_3 \\ sin\theta_3 & cos\theta_3 cos\alpha_3 & -cos\theta_3 sin\alpha_3 & a_3 sin\theta_3 \\ 0 & sin\alpha_3 & cos\alpha_3 & d_3 \\ 0 & 0 & 0 & 1 \end{bmatrix}=\begin{bmatrix} 1 & 0 & 0 & 82.5 \\ 0 & 0 & -1 & 0 \\ 0 & 1 & 0 & 316 \\ 0 & 0 & 0 & 1 \end{bmatrix}
$$

$$
{}^{3}_{4}T=\begin{bmatrix} cos4 & -sin\theta_4 cos\alpha_4 & sin\theta_4 sin\alpha_4 & a_4 cos\theta_4 \\ sin\theta_4 & cos\theta_4 cos\alpha_4 & -cos\theta_4 sin\alpha_4 & a_4 sin\theta_4 \\ 0 & sin\alpha_4 & cos\alpha_4 & d_4 \\ 0 & 0 & 0 & 1 \end{bmatrix}=\begin{bmatrix} 1 & 0 & 0 & -82.5 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}
$$

$$
{}^{4}_{5}T=\begin{bmatrix} cos\theta_5 & -sin\theta_5 cos\alpha_5 & sin\theta_5 sin\alpha_5 & a_5 cos\theta_5 \\ sin\theta_5 & cos\theta_5 cos\alpha_5 & -cos\theta_5 sin\alpha_5 & a_5 sin\theta_5 \\ 0 & sin\alpha_5 & cos\alpha_5 & d_5 \\ 0 & 0 & 0 & 1 \end{bmatrix}=\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & -1 & 0 \\ 0 & 1 & 0 & 384 \\ 0 & 0 & 0 & 1 \end{bmatrix}
$$

$$
{}^{5}_{6}T=\begin{bmatrix} cos\theta_6 & -sin\theta_6 cos\alpha_6 & sin\theta_6 sin\alpha_6 & a_6 cos\theta_6 \\ sin\theta_6 & cos\theta_6 cos\alpha_6 & -cos\theta_6 sin\alpha_6 & a_6 sin\theta_6 \\ 0 & sin\alpha_6 & cos\alpha_6 & d_6 \\ 0 & 0 & 0 & 1 \end{bmatrix}=\begin{bmatrix} 1 & 0 & 0 & 88 \\ 0 & 0 & -1 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}
$$

各相邻连杆的变换矩阵相乘可得到完整运动过程的变换关系${}^{0}_{6}T$，如下公式(3-4)所示：

$$ {}_{6}^{0}T = {}_{1}^{0}T{}_{2}^{1}T{}_{3}^{2}T{}_{4}^{3}T{}_{5}^{4}T{}_{6}^{5}T = \begin{bmatrix} n_x & s_x & a_x & p_x \\ n_y & s_y & a_y & p_y \\ n_z & s_z & a_z & p_z \\ 0 & 0 & 0 & 1 \end{bmatrix} = f(\theta_1, \theta_2, \theta_3, \theta_4, \theta_5, \theta_6, \theta_7) \quad (3\text{-}4) $$

在上式中，变换矩阵的最后一列(p_x, p_y, p_z)表示机械臂末端在工作空间中的三维坐标，需要说明的是，该坐标是以底座的世界坐标为参考系。

机械臂在进行逆解时，往往会出现不唯一解的情况，一个姿态会对应多个解。而逆向运动学则是相反，通过数值法或者解析法，根据空间中已知的末端位置和姿态，以及所有连杆的几何参数，求解机械臂相关关节的位置[52]。而求逆解的实际过程则是先计算机械臂运动学的雅可比矩阵，将关节速度映射到任务空间速度，再通过计算雅可比逆矩阵，将机械臂末端执行器的位姿误差映射到关节位置，紧接着对关节位置向量进行迭代更新来减少位姿误差[53]。

根据变换矩阵关系式和上表给出的 Franka Emika 机械臂的 D-H 参数，在 MATLAB 中利用 D-H 参数建立机器人模型，并定义各个关节的范围，连接各个连杆。再利用蒙特卡洛思想来求解 Franka Emika 机械臂的工作空间，并画出工作空间图[54]，如图 3-3 所示：

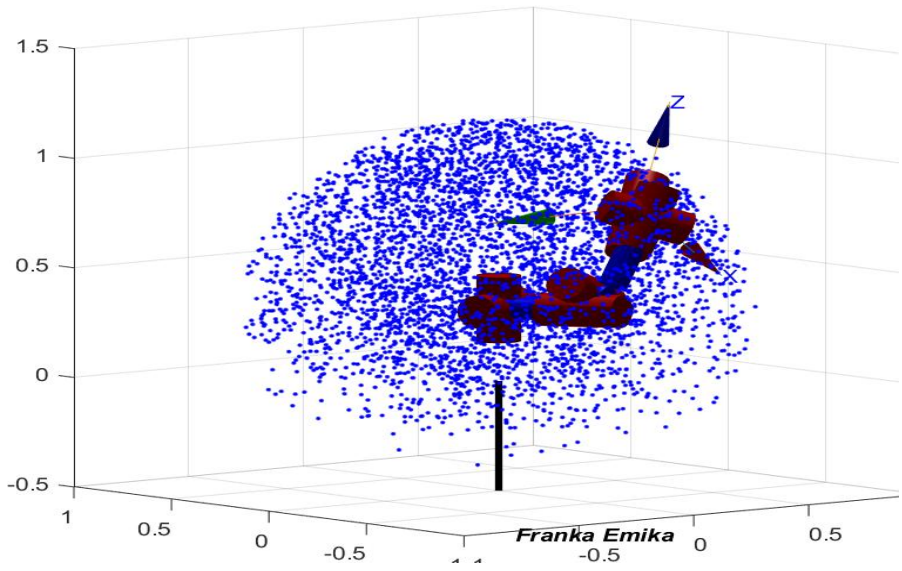


图 3-3 Franka Emika 机械臂工作空间

Fig3-3. Robotic arm linkage relationship diagram

3.3 仿真实验环境搭建

机械臂实验的仿真环境有 MuJoCo 物理引擎[55]、Pybullet 物理引擎、ROS + Gazebo 机器人仿真环境、V-REP 物理引擎。MuJoCo 物理引擎环境较难配置且代码实现不易。虽然 ROS 系统集成了 Franka Emika 环境，且对于真实机械臂的控制

很成熟，但 Gazebo 进行实验、记录数据方面比较麻烦，包括渲染都很费时间[56]。因此本文选用 PyBullet + Gym 作为实现强化学习的仿真环境。仿真环境在 Windows 系统下进行，通过第三方模块 Gym 来调用机械臂模型。深度强化学习算法采用 python 语言，使用基于 Pytorch 的框架来实现。

本文使用的 PyBullet 物理仿真器是在 Windows 下进行的，在安装 PyBullet 时可以使用 pip 指令安装[57]。PyBullet 采用客户机-服务器的接口设计逻辑，当客户机发出命令后，PyBullet 返回对应状态。内置了两种物理引擎模块，当计算机显卡算力充足时可以打开 GUI 模式，训练时可以查看机器人的运动状态。显卡算力不够时可以把可视化界面关闭，训练较为流畅。内置的 rest()函数、step()函数等可以让用户根据需求来设计状态空间、动作空间以及奖励函数等。机械臂仿真环境的设计的内容主要包括五个模块。分别为机械臂的定位和初始化、障碍物及目标点的设计、动作的执行和反馈、奖励模块的设计。

（1）机械臂的定位和初始化：首先需要获得 Franka Emika 机械臂的 URDF 文件，这是一种机器人统一描述文件，可通过 SolidWorks 进行机械臂建模转换得到[58]。同时以 Pycharm 为编程平台，将 URDF 文件导入 Pybullet 环境再设定机械臂的基座为世界坐标。根据上文利用蒙特卡洛分析得到的机械臂运动空间的范围，定义机械臂的状态空间和动作空间。状态空间根据具体需求来设计，比如障碍物的位置和速度、目标物体的位置和速度、机械臂末端执行器的位置等。动作空间则根据笛卡尔空间来定义。

（2）障碍物及目标点的设计：当机械臂导入成功后，根据本文的实验需求，在环境中导入三个小球，分别为 Ball1、Ball2、Ball3。设定三个小球的初始位置$(0, ry1, 0.3)$、$(ry2, 0.8, 0.3)$、$(ry3, -0.8, 0.3)$,其中$ry1$、$ry2$、$ry3$ 分别为(0.6,0.8)、(0.6,0.8)、(0.7,1)的随机均匀分布。这样可以使得三个小球的移动位置在 z 轴是水平的。同理，设置移动目标点的初始位置以及移动方向。同时根据 Pybullet 自带的碰撞检测 API，计算每个小球到机械臂各个连杆的最短距离。在机械臂环境中设置最大运行步数为 100。当机械臂规划次数达到 100 步时，障碍物和目标点均实现复位。

（3）动作的执行和反馈：机械臂以笛卡尔坐标系作为参考坐标系，以机械臂的末端执行器的位置为反馈，根据即时位置信息来反馈环境状态，包括机械臂的各个关节转动角度，障碍物、目标点以及末端速度、奖励值的反馈。

（4）奖励模块的设计：奖励函数的功能是根据任务需求，用多种函数约束来引导机械臂完成指定任务。如图 3-4 所示，为机械臂最终的仿真环境。

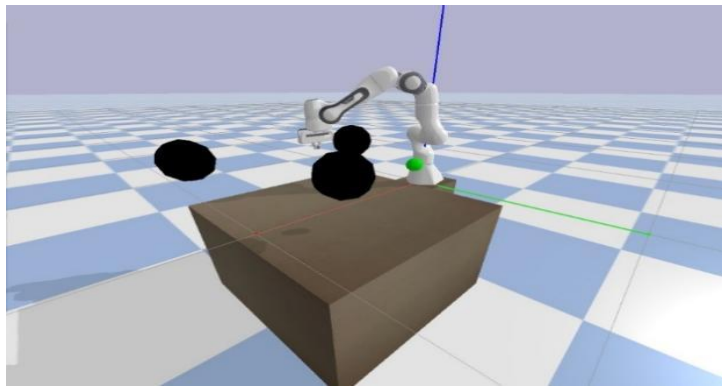


图 3-4 仿真环境

Fig3-4. Simulation environment

在实现的机械臂环境中，使用三个沿着 Y 轴方向水平移动的小球作为复杂环境下的障碍物。机械臂在自主避开障碍物的同时，末端执行器需要与绿色的移动目标点保持在一定的距离阈值之内。在动态环境中所有的物体均是随机状态。

3.4 算法模型构造

在强化学习中，奖励函数用于评估机械臂在特定环境下采取动作的好坏的评价，为智能体提供自我学习的指导信号。首先，奖励函数明确任务的目标，为机械臂完成任务提供方向指引。对于积极的信号提供正向奖励，消极信号提供负向奖励。在本文中，通过结合欧氏距离奖励函数与距离差分方法，引入实时距离指导奖励函数，提高目标接近精度。

3.4.1 状态空间和行为空间的设计

机械臂工作空间包括机械臂能够到达的三维空间及其中所能操作的位置点。机械臂的工作空间大小受到多种因素影响，主要有以下几个方面：

机械臂结构：包括机械臂的长度、关节数量、关节数量等因素对其工作空间大小有着直接的影响。越多的关节和关节数量，可以带来更多的运动自由度，从而拓展了机械臂的工作范围和覆盖面积。

关节角度范围：每个关节能够运动的角度范围也会影响机械臂的工作空间大小。角度范围较大的关节有助于机械臂覆盖更广的工作范围。机械臂制动力矩和负载能力。为了兼顾机械臂在规避移动障碍物的同时，还要追踪到移动目标点的机械臂运动轨迹，首先需要获得环境中的具体信息，包括障碍物的位置P_{obsts}和速度V_{obsts}、机械臂末端的位置P_{target}和速度V_{target}，如（3-5）所示，状态空间可以表达为元组的形式：

$$\{P_{end}, V_{end}, P_{target}, V_{target}, P_{obsts}, V_{obsts}\} \quad (3\text{-}5)$$

本文中使用的参考坐标系为笛卡尔坐标系，动作空间是以机械臂末端执行器为基准。机械臂的动作空间如下所示。当控制机械臂运动时，新动作的三个坐标乘以一个动作因子，动作定义如（3-6）所示。

$$a = \{x, y, z\} \tag{3-6}$$

3.4.2 奖励函数的设计

一般智能优化算法比如粒子群算法（Particle Swarm Sptimization, PSO）、麻雀搜索算法（Sparrow Search Algorithm, SSA）都有对应的目标函数，通过算法迭代使得目标函数值最大化[59]。而强化学习方法也属于一种优化算法，根据具体任务设计奖励函数，并通过强化学习算法实现奖励值的最大化，因此奖励函数的设计对强化学习算法的训练极为重要。

机械臂在进行自我轨迹规划时，需要做出相应的动作，并由环境对当前动作进行打分。因此奖励函数的机制是模拟人类行为，面对外界环境时，人会采取合适的行为并在环境中得到该行为的反馈。有了奖励函数的约束，机械臂的行动才会朝着期待的方向发展。奖励函数主要以机械臂的末端点到目标的欧式距离实现的，如图 3-5 所示，为机械臂在环境中的示意图：

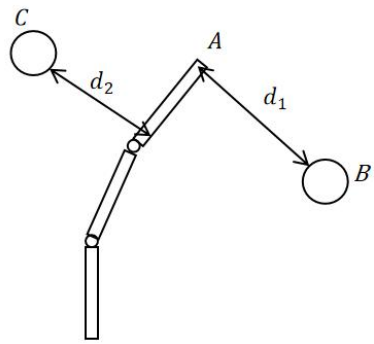


图 3-5 机械臂环境示意图

Fig3-5.Schematic diagram of dynamic environment

图中 A 点为机械臂末端执行器，B 点为目标点，C 点为障碍物。d_1为 A 到 B 的距离，d_2为 C 到 A 的最短距离。设计原则为：机械臂末端 A 点越靠近目标点，反馈的奖励值越大，反之则奖励值越小。其中公式（3-7）为目标距离奖励函数，公式（3-8）为障碍物距离奖励函数，而机械臂期望的是距离目标点最近，而远离障碍物。所以奖励函数可定义如公式（3-9）：

$$r_t = -0.5d_t^2 \tag{3-7}$$

$$r_o = -\left(\frac{\alpha}{\alpha + d_o}\right)^n \tag{3-8}$$

$$r_{original} = -w_1 r_t - w_2 r_o \tag{3-9}$$

其中d_t是机械臂末端到移动目标点的距离$distance$，r_t是奖励函数中的一部分。

得到的原始奖励函数$r_{original}$中的是r_o是机械臂与障碍物相关的奖励函数。在式子中可以看出r_o的变化趋势是：当机械臂与障碍物之间的最短距离变大，即机械臂远离障碍物时奖励值越大。当机械臂与障碍物之间的最短距离变小，即机械臂远离障碍物时奖励值越小。这样的奖励函数可以使机械臂尽可能避开障碍物，并且尽可能得靠近移动目标点。在原始奖励中设置的参数w_1=500，$w_2 = 35$，$\alpha = 1$，$n = 35$。

3.4.3 奖励函数的改进

由于原始奖励函数是单纯的稠密奖励函数，存在收敛速度慢，训练效果不好等特点[60]。而二值化奖励函数虽然可以使训练效果稳定，加快智能体学习速度和收敛性，但是单纯的二值化奖励函数过于稀疏，机械臂无法从环境中获得有效信息。二值化奖励函数如下公式（3-10）所示：

$$r_{spare} = \begin{cases} -30 & collision \\ 15 & d < a \end{cases} \tag{3-10}$$

本文设计的奖励函数在原有奖励函数的基础上引入实时指导奖励函数。在本环境中，由于机械臂跟随移动目标时，需要实现更精准的控制，因此融合指导奖励函数来引导智能体对位置进行优化，以进一步优化动作的位置选择。实时指导奖励函数中最重要的就是使用了距离差分。机械臂对移动的目标物体进行跟踪时，通过机械臂末端与目标点的上一时刻和当前时刻的距离差，作为一种奖励来辅助原始奖励函数。如图 3-6 所示，机械臂末端距离绿色目标点距离大于 d 时，则加上距离差分r_{diff}。

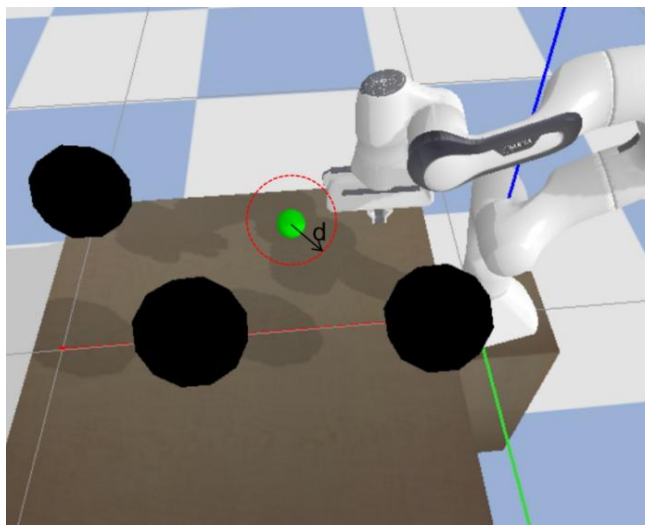


图 3-6 机械臂末端到目标点的分区

Fig3-6. Partitioning of the end of the robotic arm to the target point

通过这种实时获取环境信息的方法，使得机械臂在避开障碍物的同时，更好

得追踪目标物体。距离差分r_{diff}的公式如下（3-11）所示：

$$r_{diff} = \left|\left|d_{past} - d_{current}\right|\right|^2 \tag{3-11}$$

其中d_{past}为上一时刻机械臂末端到目标物体的距离，$d_{current}$为当前时刻机械臂末端到目标物体的距离。综上可得出最终的优化奖励函数公式(3-12)所示，根据上文可知，参数w_1=500，$w_2 = 35$，$w_3 = 35$

$$r_{ours} = -w_1 r_t - w_2 r_o + r_{spare} + w_3 r_{diff} \tag{3-12}$$

如图 3-7 所示，为奖励函数系统在机械臂自我轨迹规划算法中的流程图。机械臂将执行的动作反馈到动态环境中，再通过与环境交互获得奖励。通过奖励最大化实现自我轨迹规划的最优策略。

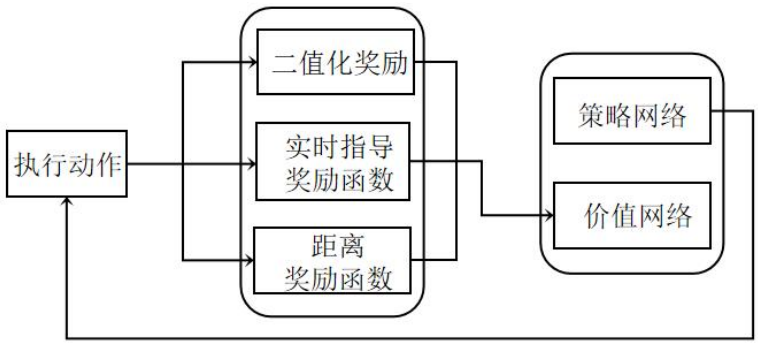


图 3-7 奖励函数算法训练示意图

Fig3-7. Schematic of reward function algorithm training

3.5 模型验证实验

模拟实验在 windows11 和 Anaconda3 平台下进行，实验所用程序基于 python3.7，神经网络基于 Pytorch3.7, 目前主流的深度学习框架，简单易懂。实验结果在 AMD Ryzen 5 5600H 和 NVIDIA GeForce GTX 1650 上获得。还需要其他的计算库来辅助，例如 Numpy 等。训练集数为 10000 集。前面第三章已经具体描述了本次实验的机械臂环境，本小节首先限定机械臂到移动目标点的距离为 0.1，三种算法使用同样的网络参数，每次训练 5000 步。

设计两组实验来验证改进后的奖励函数的优越性。对奖励函数的测试选用近端策略优化算法（PPO）和软演员评论家算法（SAC）。在实验 1 和实验 2 中，机械臂末端距离移动目标点为 0.1 和 0.05 时算成功。实验的评价指标有四种方式，分别为：成功率、平均奖励值、安全率和收敛步数，具体说明如下所示：

（1）成功率：每个回合机械臂在避开障碍物并追踪到移动目标点的成功率，是机械臂完成任务最直接的评估指标。我们选择每 20 个回合计算一次成功率。成功率和安全性越高，说明机械臂轨迹规划效果越好。成功率公式如下（3-13）所

示：

$$successful\ rate = \frac{reach_{counters}}{100\times20} \times 100\% \quad (3\text{-}13)$$

（2）安全性：机械臂每个回合内未碰撞到障碍物的概率。在这里需要说明的是，若只是安全系数高而成功率低，说明机械臂处于无效探索的状态。安全率公式如下（3-14）所示：

$$safety\ rate = 1 - \frac{collision_{counters}}{100\times20} \times 100\% \quad (3\text{-}14)$$

（3）收敛步数：成功率到达最高水平并保持平稳的最小迭代步数。收敛步数越小，说明算法收敛速度越快。

3.5.1 实验参数

为了对比分析主流算法 PPO 和 SAC 在不同环境下的机械臂自主轨迹规划中的性能，须使两种算法在网络结构上参数一致。PPO 算法具有演员 Actor 网络和评论家 Critic 网络，Actor 网络输入的是环境状态，形式与状态空间一致。PPO 和 SAC 算法的 Actor 网络输出的均是机械臂在当前状态下执行动作的概率分布。两种算法的 Actor 网络和 Critic 网络采用相同的结构，基本参数设置如表 3-2 所示：

表 3-2 两种算法的基本参数

Tab.3-2 Basic parameters of the two algorithms

	变量说明	值
相同参数	优化器（optimizer）	Adam
	奖励折扣（discount，γ）	0.99
	网络学习率	3×10^{-4}
	经验池大小（Buffer Size）	1×10^{6}
	样本个数（Batch Size）	64
	隐含层网络层数	2
	每层隐含层神经元个数	256，128
	激活函数	ReLu，tanh
	训练回合	5000
PPO	裁剪率	0.2
	优势估计（λ）	0.95
	交叉熵	0.01
SAC	平滑系数（τ）	0.01
	温度系数（α）	0.2

3.5.2 实验 1 分析

从图 3-8 可以看出实验中前 1000 个回合之内，不同奖励函数的 PPO 算法成功率差异不大，在 1000 回合到 3000 回合之间，改进后的奖励函数的成功率明显高于原始奖励函数且平滑曲线的斜率较大，表示改进的奖励函数收敛速度加快。图 3-9 表示 PPO 算法下，两种奖励函数的平均奖励值。在 1500-2000 回合之间，算法的性能提高，奖励值逐渐提高，而橙色的原始奖励函数曲线过早收敛，学习能力较弱。这与图 3-8 平均成功率的趋势相对应，且在 4000 个回合的时候，两种奖励函数下的 PPO 算法均达到了收敛的状态。

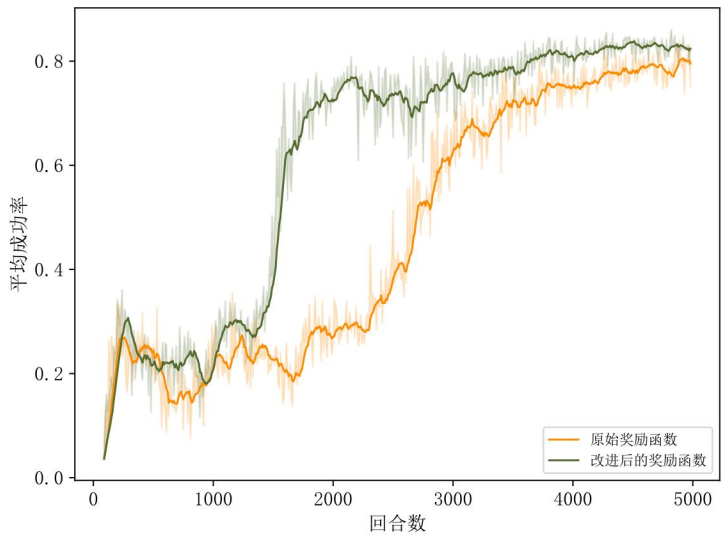


图 3-8 PPO 算法的平均成功率

Fig3-8. Average success rate of the PPO algorithm

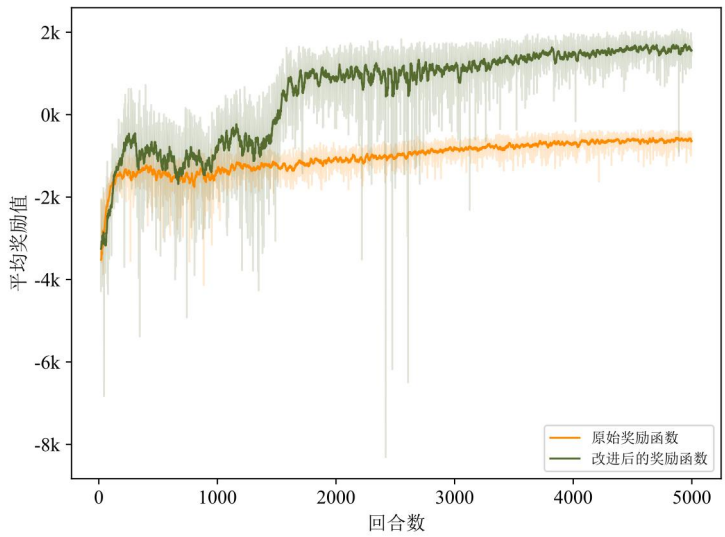


图 3-9 PPO 算法的平均奖励值

Fig3-9. Average reward of the PPO algorithm

图 3-10 代表了在两种奖励函数下，SAC 算法在 5000 个回合之内的平均成功率。可以从图中看出改进后奖励函数在训练整个过程中平均成功率均远远高于原始奖励函数，且改进后的奖励函数在 2000 回合的时候收敛了，而原始奖励函数的成功率在 3000 回合时才达到稳定收敛的状态。图 3-11 为 SAC 算法使用两种奖励函数在 5000 回合内的平均奖励值，可以看出在 1000 回合的时候，两种奖励函数的累计最大奖励值逐渐平稳，趋势与上图的平均奖励值也相吻合。

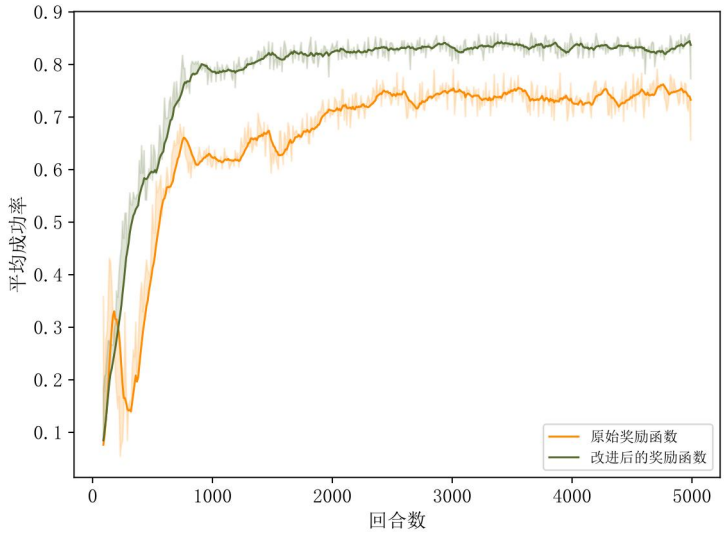


图 3-10 SAC 算法的平均成功率

Fig3-10.Average reward of the PPO algorithm

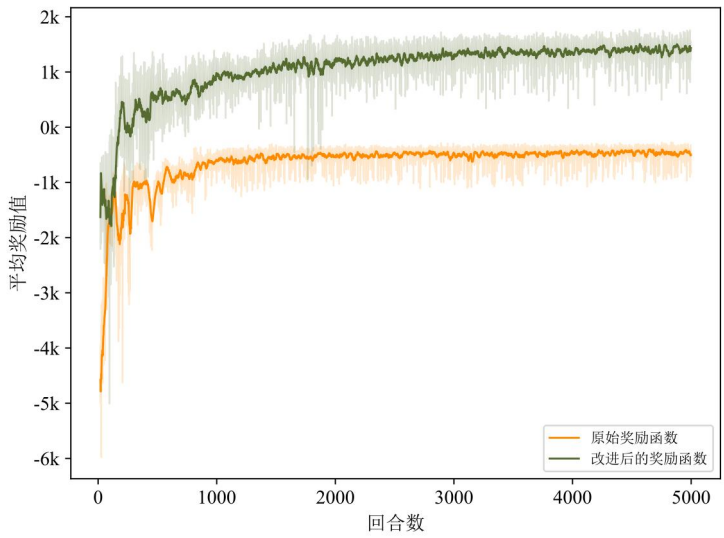


图 3-11 SAC 算法的平均奖励值

Fig3-11.Average reward of the PPO algorithm

下表 3-3 表示 2000-5000 回合之间，PPO 和 SAC 算法使用原始奖励和改进奖励函数时，每 1000 回合统计一次的成功率和安全性。可以看出在 PPO 算法中，

改进算法的成功率和安全率均高于原始算法，成功率分别高出 32.14%、7.84%、4.62%。安全率分别高出 1.74%、0.93%、1.72%。而 SAC 算法使用原始算法和改进算法的成功率和安全率，成功率有很大提升，分别为 9.74%、9.46%、9.43%，而安全性相当。可见改进奖励函数效果远远好于原始奖励函数，且 SAC 算法的结果远好于 PPO 算法。

将欧氏距离作为奖励函数的度量标准，在动态环境下可有效衡量物体之间的空间距离，但仅仅使用欧氏距离无法全面考虑到环境动态变化。因此在实验结果中机械臂追踪到移动物体并避免碰撞的任务上，完成度不高且不稳定。引入距离差分后，可以捕捉物体在连续时间步骤中的位置变化，减小欧氏距离所带来的不稳定性，使得奖励信号更加平滑和连续。

表 3-3 环境 1 中的指标分析

Tab.3-3 Analysis of indicators in Environment 1

算法名称	2000-3000		3000-4000		4000-5000	
	成功率	安全率	成功率	安全率	成功率	安全率
PPO 原始	41.78%	94.8%	71.02%	96.7%	77.92%	96.85%
PPO 改进	73.92%	96.54%	78.86%	97.63%	82.54%	98.57%
SAC 原始	73.33%	99.63%	73.91%	99.5%	74.07	99.65%
SAC 改进	83.07%	99.26%	83.37%	99.34%	83.5%	99.48%

3.5.3 实验 2 分析

实验 2 为机械臂末端距离目标点为 0.5 时，PPO 算法在 5000 回合内的训练结果。图 3-12 和 3-13 是两种奖励函数的平均成功率和平均奖励值，可以看出原始奖励函数在 2000 回合之前的平均成功率很低，在 2000-5000 回合之间，机械臂追踪目标点的成功率逐渐增高。而改进的奖励函数在 1000 回合之前的探索效率很低，平均成功率和原始奖励函数相当，在 1000-2000 回合之间成功率曲线的斜率增长速度最快，代表机械臂完成任务的成功率较快，收敛速度提高。2000-5000 回合之间，成功率增速变缓直到趋于稳定。

图 3-13 为 PPO 算法在机械臂与目标点距离控制在 0.05 时的平均奖励值。在训练的整个过程中可以看出在两种奖励函数下，PPO 算法一直在积累奖励值，并实现了奖励值的最大化。且从图中可以看出两种奖励函数的增速都在变缓慢，直到趋于平缓。说明机械臂在经过训练后，探索出了最佳的动作策略。

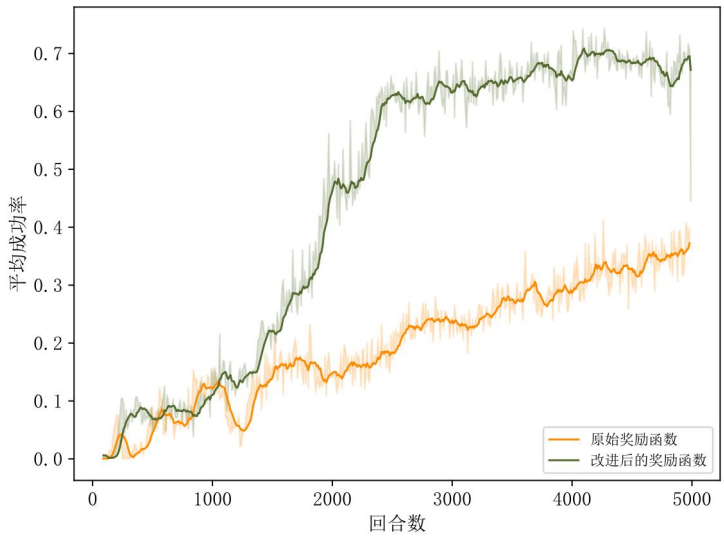


图 3-12 PPO 算法的平均成功率

Fig3-12.Average reward of the PPO algorithm

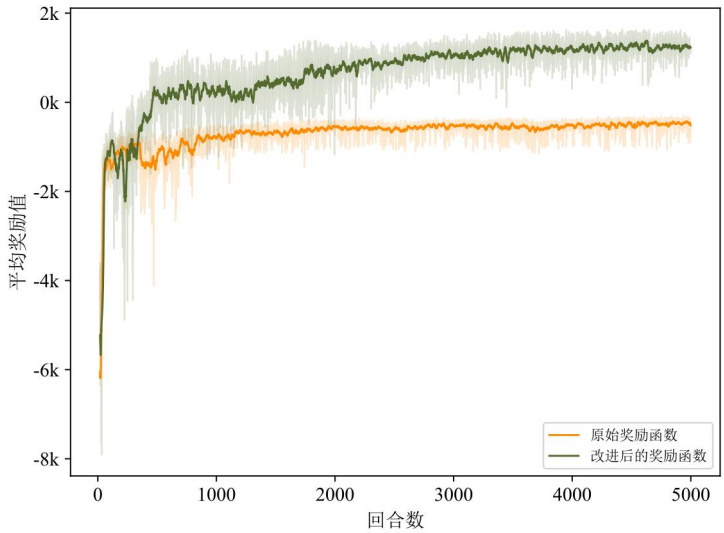


图 3-13 PPO 算法的平均奖励值

Fig3-13.Average reward of the PPO algorithm

图 3-14 和 3-15 为两种改进函数使用 SAC 算法进行训练的平均成功率和平均奖励值。从图 3-14 可以看出原始奖励函数在训练过程中成功率较低，而改进后的奖励函数在 3000 回合之内，成功率有大幅度提升。从图 3-15 可以看出原始奖励函数的平均奖励在 2000 回合时达到最大累并达到了最佳策略，对应的成功率却很低。而改进后的奖励函数在 1500 回合左右，平均奖励值最高并达到最优策略。

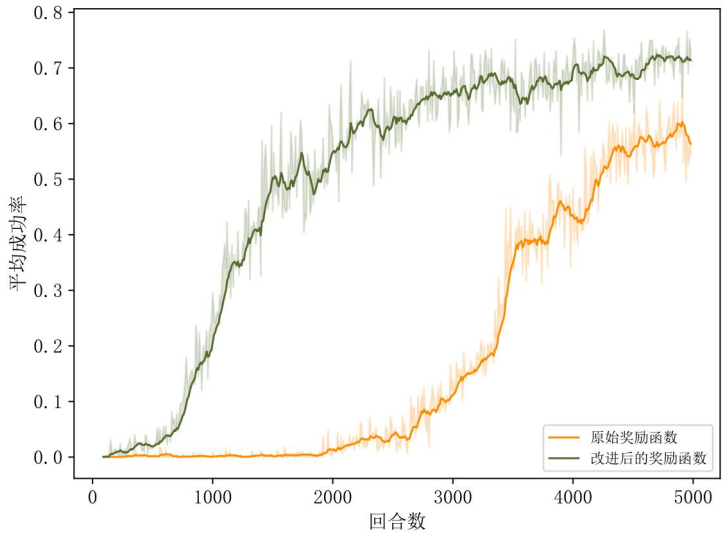


图 3-14 SAC 算法的平均成功率

Fig3-14.Average reward of the PPO algorithm

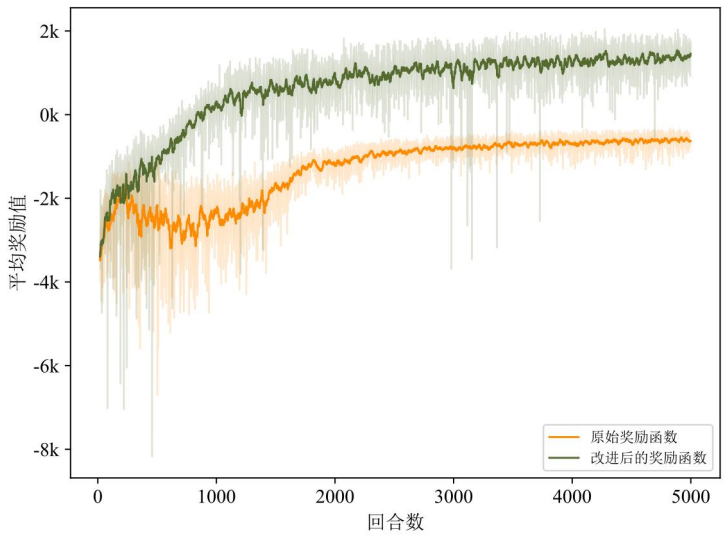


图 3-15 SAC 算法的平均奖励值

Fig3-15.Average reward of the PPO algorithm

表 3-4 是在实验 2 下 2000-5000 回合中，两种奖励函数使用 PPO 和 SAC 算法，每 1000 个回合计算的成功率和安全率。可以看出改进后的奖励函数使用 PPO 算法训练时成功率相对于原始奖励函数，分别提高了 38.15%、38.94%、35.08%，安全率处于相当的水平。使用 SAC 算法训练时的成功率相对于原始奖励，分别提高了 51.15%、36.32%、16.45%。通过分别在两种环境下，使用 PPO 算法和 SAC 算法作为基准强化学习算法，对比改进奖励函数和原始奖励函数的性能。SAC 算法下的成功率和安全性均得到保障，且算法收敛速度较快，因此在接下来的算法改进中会使用 SAC 算法作为基本算法。

表 3-4 环境 2 中的指标分析

Tab.3-4 Analysis of indicators in Environment 2

算法名称	2000-3000		3000-4000		4000-5000	
	成功率	安全率	成功率	安全率	成功率	安全率
PPO 原始	5.11%	99.31%	30.701%	99.5%	54.03%	99.4%
PPO 改进	57.85%	98.74%	65.56%	99%	68.67%	99.2%
SAC 原始	19.7%	99.57%	26.62%	99.51%	33.59%	99.8%
SAC 改进	61.26%	99.4%	67.02%	99.3%	70.48%	99.3%

3.6 本章小结

本章首先分析了机械臂的运动学，并根据本文所使用的机械臂 Franka Emika 的 D-H 参数及逆行机械臂运动学分析，然后根据环境的特点选择 PyBullet 物理引擎建模。给出了动态环境中的原始奖励函数，并改进了原有奖励函数，最后分别在 PPO 和 SAC 算法为基础下，对两种奖励函数进行测试。实验结果表明我们的改进奖励函数的效果远远高于原始奖励函数，在成功率、奖励值和安全性都均有提升。但 PPO 训练不稳定，成功率的起伏比较大。因此我们选用 SAC 算法作为后续改进的基础算法。

第 4 章 改进 SAC 算法的机械臂自主轨迹规划

4.1 引言

本章的工作是基于第三章基础上进行进一步研究与改进。在第三章对 PPO 和 SAC 算法使用改进奖励函数进行机械臂轨迹规划的实验分析，虽然改进效果有所提升，但由于处于复杂的动态环境中，SAC 算法无法完全观测外界环境，导致初始阶段出现的机械臂自主轨迹规划训练效果差的问题。针对以上问题，首先提出基于改进 SAC 算法（AL-PER-SAC）的机械臂轨迹规划流程，然后开展对 SAC 算法的改进工作，最后实验结果分析机械臂自主轨迹规划的效果。

4.2 注意力机制

在进行深度学习时，模型的学习往往要处理海量的信息，因此在处理信息时，数据和数据之间有某种联系或者部分数据是相对重要的。因此这时候注意力机制就极为重要。如图 4-1 所示，注意力机制可以通过对输入数据进行重要性分配，凸显出部分数据的重要性，有助于模型在处理大量信息时集中注意力于相关特征，进而提高模型的整体表现[61]。使用注意力机制时往往需要一个查询向量 Q，然后通过一个打分函数计算查询向量和输入之间的关联性，得到对应的打分，再使用 Softmax 对分数进行归一化。归一化的作用是使得查询向量 Q 在各输入上的重要性分布，对应公式如下（4-1）所示：

$$a_i = softmax\big(s(x,q)\big) = \frac{\exp\big(s(x,q)\big)}{\sum_{j=1}^{n}\exp\big(s(x,q)\big)} \tag{4-1}$$

其中x代表输入，q是查询向量。最后需要将这些重要性分布有选择得去提取重要信息。可以对重要性分布进行加权求和，得到的结果体现了模型应该关注的内容。这个结果 context 公式如下(4-2)所示：

$$context = \sum_{i=1}^{n} a_i x_i \tag{4-2}$$

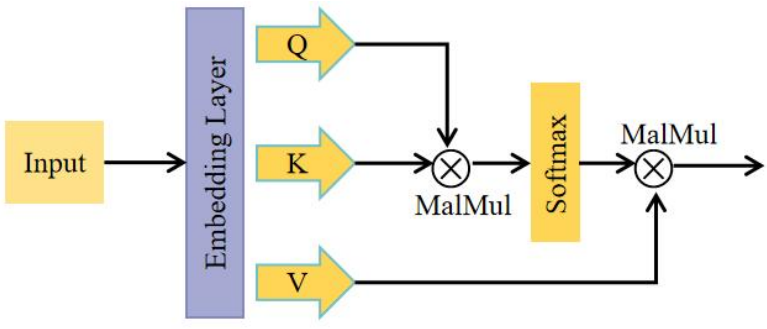


图 4-1 注意力机制结构

Fig4-1.Structure of attention mechanism

对于计算方式有加性模型、点积模型和缩放点积模型等。点积模型相对于加性模型具有更好的计算效率。且当输入向量的维度比较高的时候，点积模型通常有比较大的方差，从而导致 Softmax 函数的梯度会比较小。

4.3 长短期记忆神经网络

LSTM 网络的内部结构包括遗忘门、输入门、输出门。结构图如图 4-2 所示，遗忘门用于控制上一个时间戳的状态向量c_{t-1}对当前时间戳的影响[62]。遗忘门的控制变量g_f如公式（4-3）所示，经过遗忘门处理后，LSTM 的状态向量变为$f_t c_{t-1}$。输入门，用于控制 LSTM 结构对于输入的接收程度，通过对当前输入x_t和上一个时间戳的h_{t-1}作线性变换得到新的输入向量$\tilde{c}_t$，通过输入门控制接受输入的量。

更新门用于刷新 Memory，在遗忘门和输入门的控制下，LSTM 可以以有选择地读取上一个时间戳的c_{t-1}和当前的新输入$\tilde{c}_t$，状态向量c_t如下公式（4-4）所示。

输出门在 LSTM 结构内部，状态向量不会全部输出，而是在输出门的作用下有选择地输出。输出门控变量如下公式（4-5）所示。当输出门$g_0 = 1$ 时，输出通道打开，LSTM 的状态向量c_t全部用于输出。反之，则无法用作输。

$$f_t = \sigma\left(w_f[h_{t-1}, x_t] + b_f\right) \tag{4-3}$$

$$\tilde{c}_t = \tanh\left(w_c[h_{t-1}, x_t] + b_c\right) \tag{4-4}$$

$$h_t = g_0 \cdot \tanh\left(c_t\right) \tag{4-5}$$

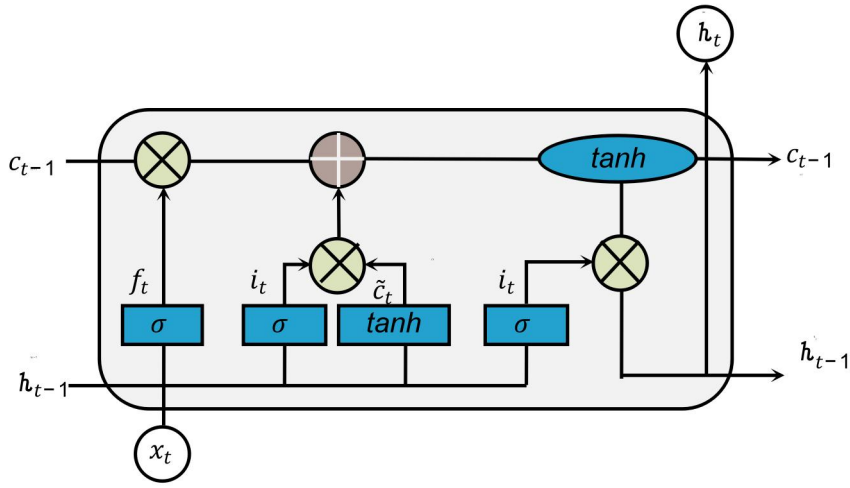


图 4-2 长短期记忆神经网络结构

Fig4-2.Structure of Long Short Term Memory Neural Network

其中，w_f和和b_f为遗忘门参数，σ为激活函数。w_c和和b_c为输入门参数，h_{t-1}为输出，x_t为输入，g_0为输出门控制变量。

4.4 SAC 算法的改进

4.4.1 Attn-LSTM 模块的引入

机械臂通过轨迹与环境互动，每个轨迹都按顺序由状态、动作、奖励和下一个状态组成。从初始状态开始，每一轮都会产生动作和奖励，并使奖励最大化，最终目标是在整个训练过程中使奖励最大化。执行一个动作会导致机械臂过渡到另一个状态，这是机器人运动学的一种表现形式。环境会对机器人的行动提供反馈，机器人会获得奖励，奖励的标准由环境中的奖励函数决定，包括任务的完成情况和运动的稳定性。机器人经过训练，从历史经验中学习优化策略，使累积奖励值最大化，直至完成给定任务。机器人通过与环境的持续互动来实现这一目标。

当机械臂在动态环境中进行自主轨迹规划时，通常单一的状态已经无法提供足够的信息来引导机械臂做出决策。复杂的环境中，机械臂会关注所有的动态信息，导致无法做出合理有效的决策。比如使用强化学习算法进行训练时，机械臂会为了尽量避开障碍物而做无效的探索。深度学习中的 Attention 机制具有增强重点信息、降低干扰和冗余信息，可以选择性关注环境中更重要的障碍物和目标点的信息。由于障碍物和目标的位置是移动的，因此作为输入信息直接输入会导致模型无法观测到完整的状态信息。

因此本文研究先在 SAC 算法的策略网络和评价网络中引入注意力机制模块，使得机械臂能够关注到重要信息。然而由于动态环境因素的影响，单纯的 MLP 网络只能根据状态和环境进行线性拟合，且机械臂在执行任务，进行自主轨迹规划的过程不仅与当前状态有关，还与历史运动信息相关。在研究进展中提到有研究人员利用 LSTM 网络来预测移动障碍物的位置，且取得了较好的效果。因此，我们研究将注意力机制 Attention 和 LSTM 网络进行融合，设计了注意力机制下的深度长短期记忆网络（Attn-LSTM）来改进 SAC 网络。

如图 4-3 所示，为 Attn-LSTM 结构图。其中，注意力机制接收状态信息，将机械臂环境中的信息进行关键特征的有效描述，再经过 LSTM 记忆过去的信息来适应环境的动态环境的变化，帮助机械臂学习到更具表达力的策略。其中输入状态的个数为 30，全连接层为 256，经过注意力机制的处理再输入到 LSTM 网络计算，得到输出信息，其中 LSTM 网络的隐含层为 256。输出信息则是策略网络和价值网络的输入，通过对输入状态进行加权，实现对输入信息更精准的处理。

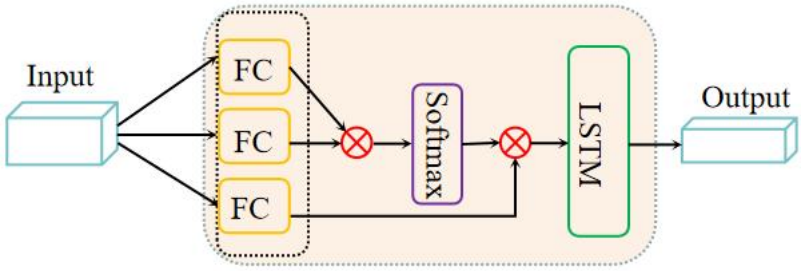


图 4-3 Attn-LSTM 结构图

Fig4.3 The constructure of Attn-LSTM

图 4-4（a）为价值网络结构，它的输入是状态和动作。经过 Attn-LSTM 结构来提取输入信息的特征，深度神经网络可以实现每个状态-动作到估计值的映射。其中策略网络的隐含层均为全连接层，神经元个数分别为 256、128，将状态输入后使用 Relu 激活函数，实现状态特征的更好提取。策略网络的第三层神经元个数为 3 个，输出均值和标准差，再利用高斯分布进行采样，最后使用 Tanh 激活函数输出动作。价值网络则是三层的全连接层，神经元个数分别为 256、128、1，全连接层之间均使用 ReLu 函数，最终输出价值 Q 用于策略网络的更新。图 4-4（b）策略网络结构，输入信息$\{P_{end}, V_{end}, P_{target}, V_{target}, P_{obsts}, V_{obsts}\}$作为输入状态，Attn-LSTM 的输出被输入到全连接的深度神经网络中，接着生成一个动作,然后根据价值网络输出计算估计动作的价值 Q。

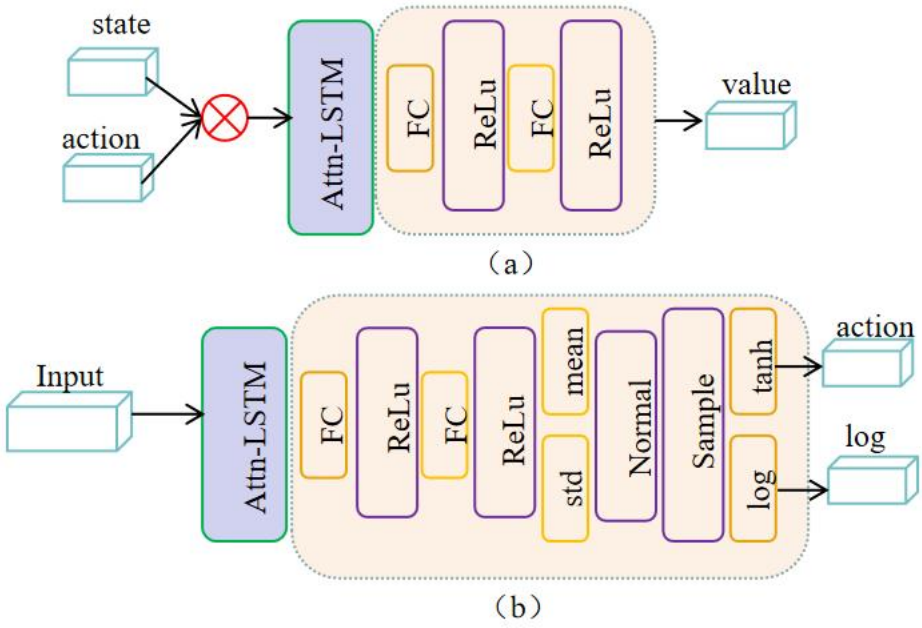


图 4-4 价值网络（a）和策略网络结构图（b）

Fig4.4 The constructure of critic network (a) and actor network (b)

4.4.2 优先级经验回放

与深度学习不同的是，智能体与环境交互产生的数据会以数组的形式存储经

验回放池中，因此数据的获取是缓慢的过程。且采集数据在时间上是连续的，因此数据样本之间会有较强的相关性[63]。重用经验是从经验池中均匀采样，忽略了经验的重要程度，而随机采样可以破坏数据之间的相关性，更好得利用数据。SAC 算法采取的是随机采样进行训练的，因此数据利用性较强。但在动态环境中，由于环境状态是实时变化的，因此随机采样技术在一定程度上可以提高数据的利用率，但利用效率还有待提高。上节提到的注意力机制是通过对数据重要性重新分配，实现对数据的更好利用。优先级经验回放也是类似的作用，首先保持经验池的队列结构，根据先进后出的原则，当长度达到设定值时开始剔除数据。通过优先采集经验池中具有较高价值的数据样本，帮助算法更快得收敛，加速智能体训练。

在 DQN 算法的核心公式中，目标价值 Q 的更新是当前 Q 值与 TD-error 的和。因此 TD-error 越小越好，当 TD-error 比较小时，就意味着我们当前的 Q 函数离目标的 Q 值差别很大，需要多次进行更新。如下公式（4-6）所示，为 TD-error 的公式：

$$\delta_j = r(s_t, a_t) + \gamma Q^{'}(s_{t+1}, a_{t+1}, \omega) - Q(s_t, a_t, \omega) \tag{4-6}$$

TD-error 是反应价值网络估计值准确与否的关键指标，当 TD-error 的绝对值越大，表示估计值的偏离程度越大。在训练过程中，机械臂会因 TD-error 过大而产生错误的行为，结合强化学习的本质（智能体通过不断试错从而达到学习的目的），优先级经验采样使得机械臂避免重复执行错误行为，经过反复多次采样经验池中的经验元组，从而提升整体的训练效果。

通过概率方式抽取经验，保证经验被抽到的随机性。令每条经验的优先值如公式（4-7）所示：

$$\mathrm{P(i)} = \frac{\mathrm{p_i}}{\sum \mathrm{p_i}} \tag{4-7}$$

其中$\mathrm{p_i} = |\delta_t + \varepsilon|$, ε是比较小的数值。目的是为了保证当 TD-error 较小时，也能被抽到。还需要加入重要性采样来调整学习速率，公式如（4-8）所示：

$$w_i = (\frac{1}{N} \cdot \frac{1}{P(i)})^{\beta} \tag{4-8}$$

N 表示经验回访池的容量，β是超参数。在本文中，首先再 SAC 算法的策略网络和价值网络中引入 Attn-LSTM，再加入优先级经验回放来优化算法采样能力，提出 AL-PER-SAC 算法。具体的流程如图 4-5 所示。当机械臂与环境进行交互时，得到的数据是经验元组$(\mathrm{s_t}, \mathrm{a_t}, \mathrm{r_t}, \mathrm{s_{t+1}})$,再将经验存在经验池中，机械臂与环境持续交互产生样本数据，再经过优先级采样获取重要数据样本。SAC 算法根据优先级

经验采样，将价值量较高的经验元组从经验池中采样再进行训练。

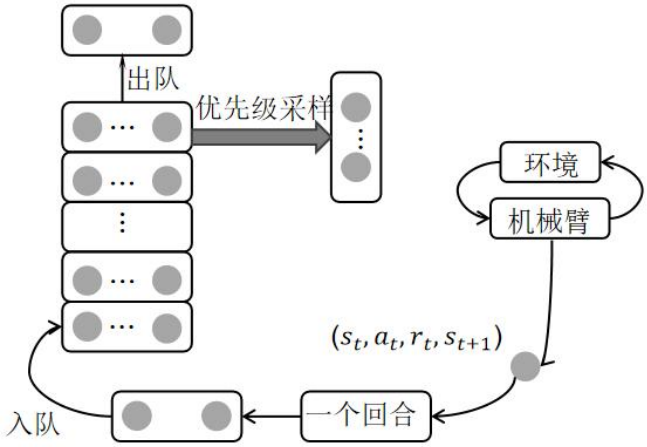


图 4-5 优先级经验采样流程

Fig4.5 Priority Experience Sampling Process

4.5 机械臂自主轨迹规划流程

如图 4-6 所示，为 AL-SAC 的算法结构图。根据上节对 SAC 算法的策略网络、Q 值网络和价值网络中加入 Attn-LSTM 结构，其中 Actor 网络和 Critic 网络是改进之后的，同时需要将改进后的奖励函数加入到机械臂环境中作为基准奖励函数，最后再进行机械臂的自主轨迹规划。

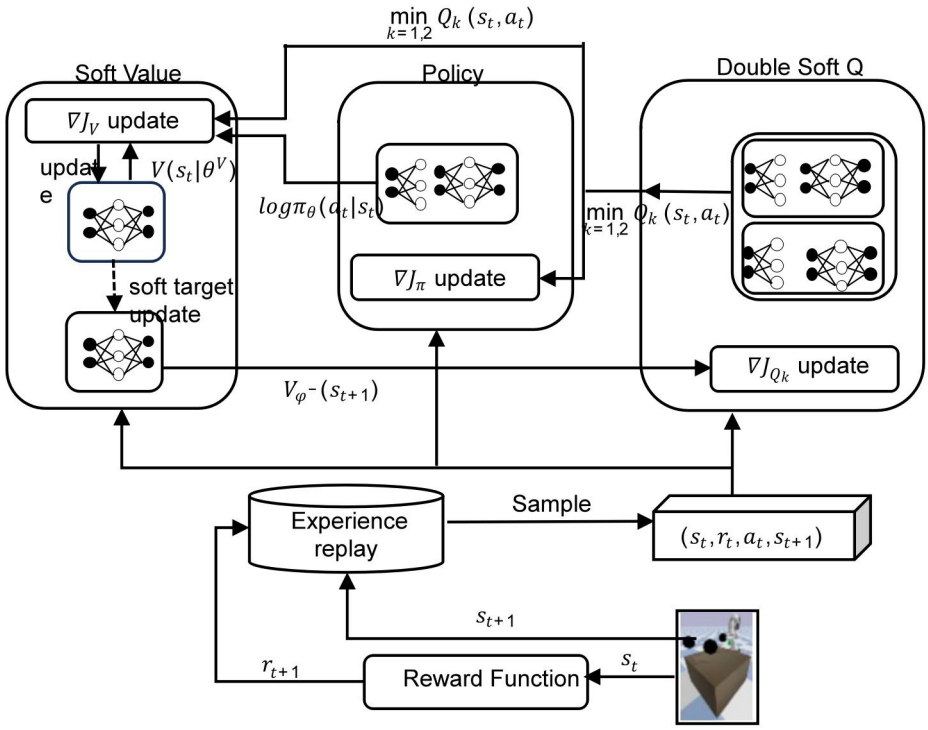


图 4-6 基于改进 SAC 的算法结构图

Fig4.6 Structure of the algorithm based on improved SAC

首先在训练之前，机械臂从环境中获得输入初始状态信息s_t,经过改进后策略网络的计算得到动作a_t，然后动作a_t和状态信息s_t作为价值网络的输入，得到当前动作的估计值。同时动作a_t作为价值网络的输入，并且反向更新价值网络。机械臂将动作a_t反馈到环境中，奖励函数根据当前动作评价任务完成度，并反馈动作所对应的奖励值。在这个过程中，当前状态s_t、下一状态s_{t+1}、当前动作a_t和奖励值r_t,一起组成一个四元数组(s_t, a_t, r_t, s_{t+1})存储在经验回放池中。接着从经验回放池中通过优先级经验采样来采样高价值数据样本，对网络进行更新。其中 SAC 树算法虽然采取的是随机抽取的策略，在一定程度上提高了数据的利用率，但针对环境中样本重要性未做区分。机器人通过与环境的持续互动来实现这一目标，随着奖励值的累计最大化、策略网络的学习能力提高，机械臂采取的动作更合理。价值网络不参与策略网络和评价网络的实施，它的作用是通过状态反向更新策略网络和价值网络。

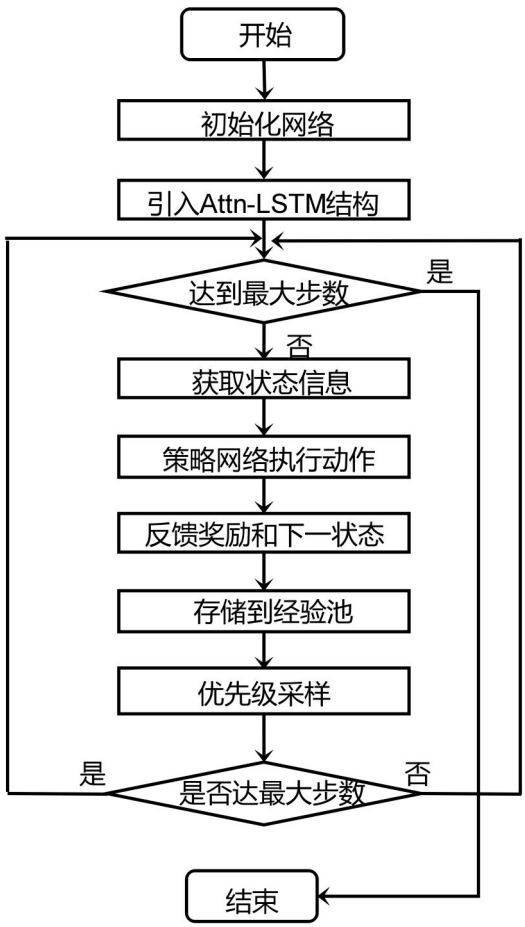


图 4-7 算法整体流程图

Fig4-7.Overall algorithm flowchart

如图 4-7 是本次算法设计的整体流程，如图所示。在每个回合开始时，PyBullet

环境中有最大步数限制，当机械臂在一个回合的运动步数超过最大步数时就会重置环境，机械臂回到设定的初始状态。机器人依靠环境中定义的状态信息，输入到 AL-PER-SAC 策略网络计算并输出动作。与环境交互后，会反馈当前动作对应的奖励值和下一状态。将获得的经验存储在经验池中，再通过优先级经验回放将价值量高的经验采样给机械臂训练。表 4-1 为改进的 SAC 算法的伪代码。

表 4-1 改进的 SAC 算法的伪代码

Tab.4-1 Pseudo-code for the improved SAC algorithm

算法：AL-PER-SAC

初始化评价网络的参数 θ_1, θ_2 ,策略网络的参数 θ_π ,注意力机制参数 θ_{attn} 和LSTM网络的参数θ_L

复制参数 $\theta_1^{'} \leftarrow \theta_1,\ \theta_2^{'} \leftarrow \theta_2$

For episode $i < N$ **do**

  $s_t = LSTM.(attention.(input))$

  选择动作 $a_t = \pi_\theta(s_t)$

  执行动作 (s_t, a_t, r_t, s_{t+1})

  保存经验池 $R \leftarrow (s_t, a_t, r_t, s_{t+1})$

    for $k < M$ **do**

      赋予经验池中数据优先级$\mathrm{P_t} = \max_i \mathrm{P_i}$

      根据优先级$\mathrm{P_t}$从经验池中采样数据样本 R

      计算每条经验的重要性权重ω_i

      $y_i = r_i + \gamma min_{j=1,2} Q_{w_j^-}(s_{i+1}, a_{i+1}) - \alpha log\pi_\theta(a_{i+1}|s_{i+1})$

      更新每个评价网络:

        最小化损失函数 $L = \frac{1}{N}\sum_{i=1}^{N}(y_i - Q_{w_j}(s_{i+1}, a_{i+1}))^2$

      通过TD-error更新优先级$\mathrm{P_t}$

      更新每个策略网络:

        $L_\pi(\theta) = \frac{1}{N}\sum_{i=1}^{N}(\alpha log\pi_\theta(\tilde{a}_i|s_i) - \min_{j=1,2} Q_{w_j}(s_i, \tilde{a}_i))$

      更新因子 α

      更新目标评价网络:

        $\theta_{1,2}^{'} \leftarrow \tau\theta_{1,2} + (1-\tau)\theta_{1,2}^{'}$

    end for

end for

4.6 实验参数设置

在实验阶段有三个移动的黑色障碍物和一个移动的绿色目标点。机器人在避开障碍物的同时向目标点移动。同时每 20 次计算平均成功率、安全率和平均奖励。

利用我们的奖励函数作为 LSTM-SAC 、AL-SAC 和 AL-PER-SAC 算法的奖励函数，在两种环境中评估机械臂的自主轨迹规划的性能。

表 4-2 为 Attn-LSTM 结构的网络参数，本文使用的 SAC 算法使用了三种不同的神经网络，分别称为行为者网络、批评者网络和价值网络。行为者网络和价值网络的结构相同，SAC 算法的结构如表 4-3 所示

表 4-2 Attn-LSTM 的结构

Tab.4-2 Structural of the Attn-LSTM

网络层	策略/价值网络		评价网络	
	数量	激活函数	数量	激活函数
输入层	30	ReLu	30+3	ReLu
编码层	256	ReLu	256	ReLu
LSTM 层	256		256	

表 4-3 SAC 算法的结构

Tab.4-3 Structure of the SAC algorithm

网络层	策略/价值网络		评价网络	
	数量	激活函数	数量	激活函数
输入层	256	ReLu	256	ReLu
隐含层 1	256	ReLu	256	ReLu
隐含层 2	128	ReLu	128	Linear
输出层	3	tanh	1	

其中基准算法 SAC 算法的超参数如表 4-4 所示：

表 4-4 SAC 算法的超参数如表

Tab.4-4 Hyperparameters of the SAC algorithm

超参数名称	符号表示	值
点积注意力机制缩放因子		16
策略网络学习率	α_A	0.0005
评价网络学习率	α_C	0. 0005
价值网络学习率	α_V	0.0005
批数据大小		256
软更新参数	τ	0.005
经验回放池容量		1000000
折扣率	γ	0.99
奖励值缩放因子		2

4.7 实验结果分析

在搭建的实验仿真环境中，如图 4-8 所示，在（a）中机械臂最初处于初始状态，接着（b）中障碍物移动到机械臂的左侧，（c）中机械臂避开了左侧的障碍物并且向着移动目标点靠近，机械臂的末端执行器要追踪到移动目标点且精度保持在一定范围内，（d）中机械臂末端成功追踪到移动目标点，机械臂轨迹规划是自

主完成的过程。实验一和实验二分别在环境 1 和环境 2 中使用 LSTM-SAC 算法作为基线，要求机器人在不与障碍物碰撞的情况下，将机器人机械手末端与移动目标点之间的距离分别保持在 0.1 和 0.05 以内。

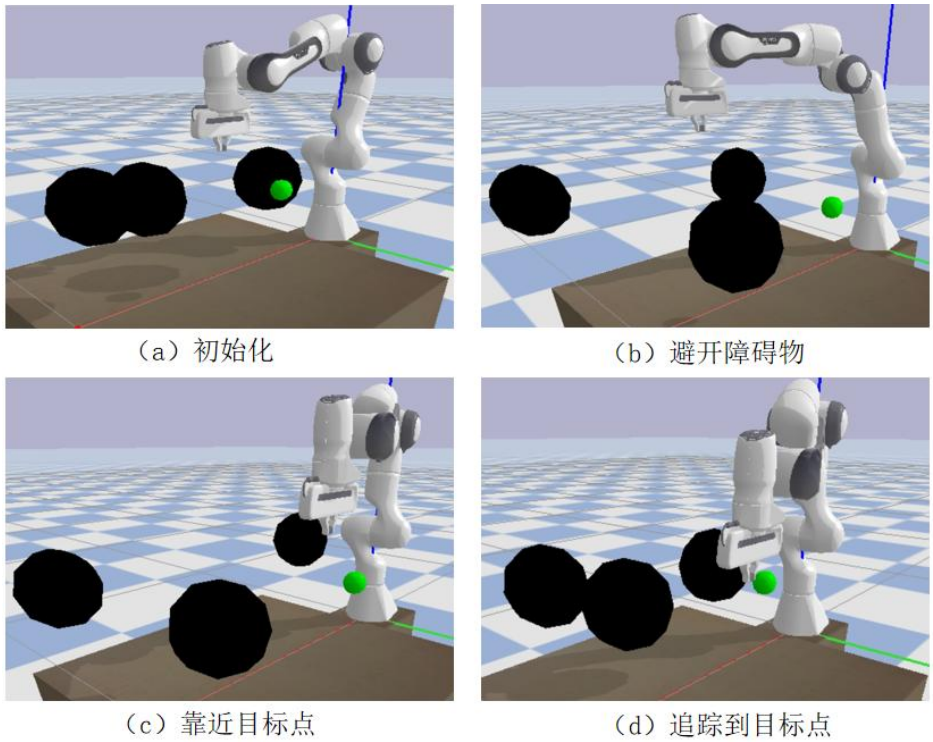


图 4-8 控制实验规划过程

Fig4.8 Control the experimental planning process

4.7.1 环境 1 对比实验

图 4-9 在环境 1 下进行，表明了三种算法在训练 5000 步的成功率。可以看出使用 LSTM 的算法训练时成功率上升最慢，改进算法 AL-PER-SAC 收敛最快，且成功率略有提高，成功率在 0-400 个回合之间增速最快，在 400-1000 回合之间增速逐渐变缓慢，1000-5000 回合之间趋于平稳。使用 AL-SAC 算法的成功率在 0-400 回合之间增速略小于 AL-PER-SAC 算法，在 400-500 回合之间成功率出现下滑，在 500-1000 回合之间又出现缓慢增长。以 LSTM-SAC 为基准算法可以看出，使用改进算法，整体的成功率有所提高，训练过程中最稳定，且以训练的回合数为基准，成功率的收敛速度最快。

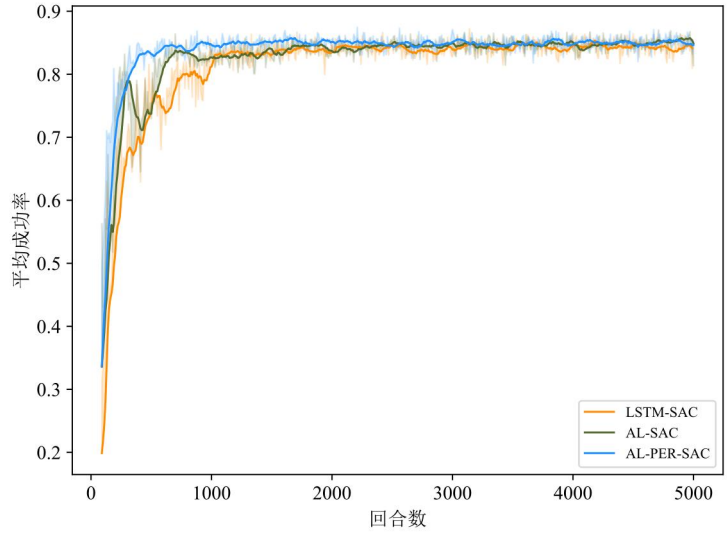


图 4-9. 环境 1 中几种算法成功率对比图

Fig4.9 Comparison of the success rates of several algorithms in Environment 1

图 4-10 对图例的描述与图 4-9 一致，代表了三种算法的平均奖励值。其中使用 AL-SAC 算法在训练过程中，奖励值略高于 LSTM-SAC 算法。我们的算法总奖励值在三种算法中处于最高。可以看出 AL-SAC 算法的平均奖励在 200-1000 个回合时，相对于其他两种算法的曲线更加平稳。优先级经验回放的加入使得算法的收敛速度提升，训练效果更好。针对机械臂轨迹规划问题，算法的平稳性更重要，证明了 AL-PER-SAC 算法的效率更优。

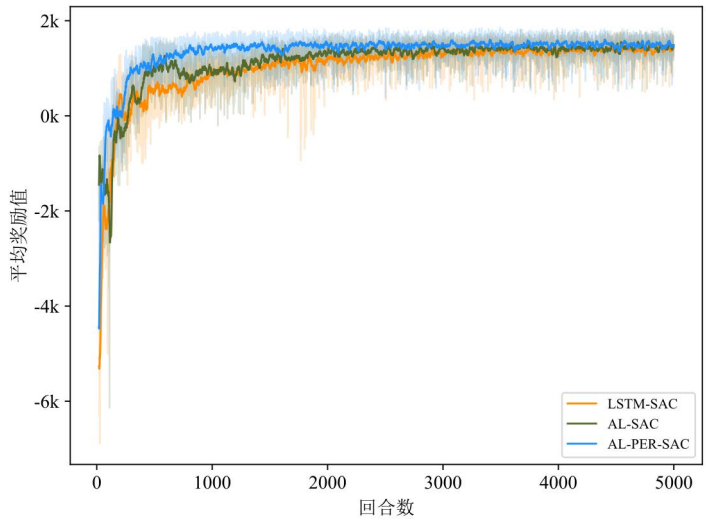


图 4-10. 环境 1 中几种算法奖励值对比图

Fig4.10 Comparison of reward values for several algorithms in Environment 1

表 4-5 是在环境 1 中，三种算法前 3000 回合的成功率、收敛步数和安全率的表现。可以看出 AL-SAC 算法相对于 LSTM-SAC 算法，在成功率上分别提高了

8.56%、3.88%和 0.78%，而收敛速度提前了 400 步。AL-PER-SAC 算法相比 AL-SAC 算法，在成功率上提升了 16.56%、2.26%、1.98%，收敛步数提前了 200 步。比较三种算法的平均成功率、收敛步数和安全率，可以看出改进算法 AL-PER-SAC 在避开障碍物的同时，仍然保持着最高的追踪成功率，且收敛速度最快。可以看出在环境 1 中 AL-PER-SAC 进行自主轨迹规划的效果最佳。

表 4-5 环境 1 中三种算法评价指标

Tab.4-5 Three Algorithm Evaluation Indicators in Environment 1

算法名称	0-1000	1001-2000	2001-3000	0-3000	0-3000
	成功率	安全率	成功率	安全率	成功率
LSTM-SAC	53.72%	78.34%	82.29%	1400	0.02
AL-SAC	62.28%	82.22%	83.07%	1000	0.017
AL-PER-SAC	78.84%	84.48%	85.05%	800	0.0093

为了进一步验证 AL-PER--SAC 模型的性能，本文将收敛后的三个模型在具有四个移动障碍物环境中进行 2000 次测试，其中机械臂需满足实验一的条件（机械臂末端距离目标点的距离不超过 0.1）。测试结果如表 4-6 所示，其中三种算法安全系数都相当，都能避开障碍物，但 LSTM-SAC 算法追踪到目标点的成功率仅为 77.57%，AL-SAC 算法和 AL-PER-SAC 算法的成功率分别为 79.81%和 83.64%。可以看出 AL-PER-SAC 算法相比其他两种，在进行自我轨迹规划时的效率相对更高，无效探索更少。

表 4-6 2000 次测试中三种算法的性能比较

Tab.4-6 Performance comparison of three algorithms in 2000 tests

算法名称	成功率	安全率
LSTM-SAC	77.57%	98.9%
AL-SAC	79.81%	99.2%
AL-PER-SAC	83.64%	99.2%

4.7.2 环境 2 对比试验

图 4-11 表示环境 2 中三种算法的平均成功率。可以看出 LSTM-SAC 算法的成功率在 0-3000 个回合之间缓慢上升，AL-SAC 算法和 AL-PER-SAC 算法均有较快的收敛速度，在 1000 个回合的时候达到成功率最高点，但改进算法 AL-PER-SAC 相对于 AL-SAC 算法收敛速度更快。

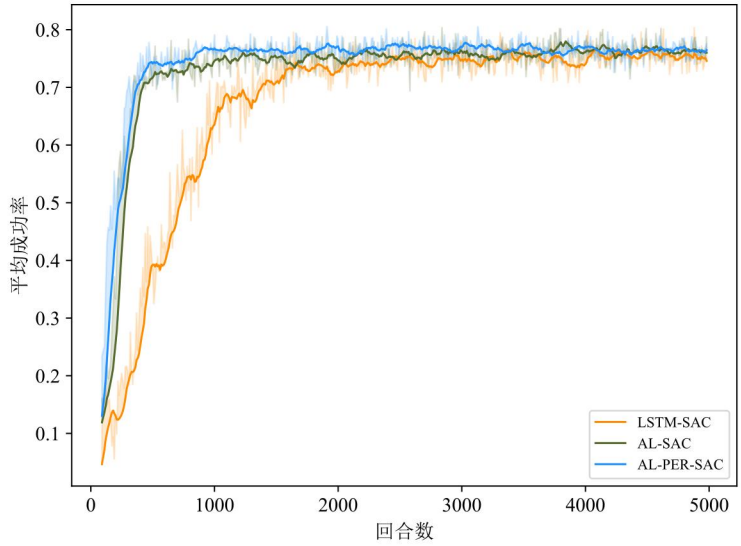


图 4-11. 环境 2 中几种算法成功率对比图

Fig4-11 Comparison of successful rate for several algorithms in Environment 2

图 4-12 为环境 2 下三种算法的平均奖励值，通过奖励值变化的趋势验证 AL-PER-SAC 算法的可靠性。可以看出在前 200 回合之间，三种算法持平，奖励值相当。但在 200-1200 之间逐渐拉开差距，AL-PER-SAC 算法在 200-1200 之间奖励值上升最快，最终的总奖励值也是最高的，结合成功率，表明算法可以成功帮助机械臂规划出实验的最佳轨迹，通过实验结果可以验证改进算法可以提升对机械臂的轨迹规划的效率。

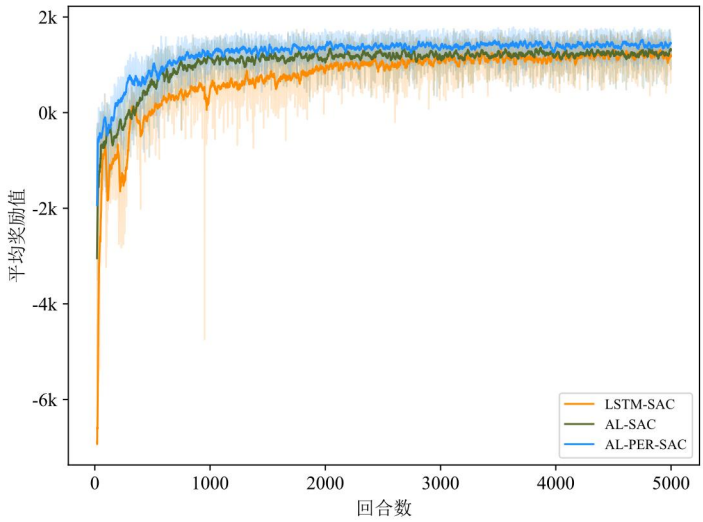


图 4-12. 环境 2 中几种算法奖励值对比图

Fig4-12 Comparison of reward values for several algorithms in Environment 2

表 4-7 是在环境 2 下三种算法成功率、收敛步数和安全率方差的评价指标。AL-SAC 算法每 1000 步之内相对于 LSTM-SAC 算法，成功率分别提升了 13.74%、

5.3%、3.21%，收敛步数提前了 400 步。而 AL-PER-SAC 算法相对于 AL-SAC 算法，成功率提高了 17.56%、18.48%、7.19%，收敛步数提前了 1600 步。

表 4-7 环境 2 中三种算法评价指标

Tab.4-7 Three Algorithm Evaluation Indicators in Environment 2

算法名称	0-1000	1001-2000	2001-3000	0-3000	0-3000
	成功率	安全率	成功率	安全率	成功率
LSTM-SAC	18.93%	51.33%	66.08%	3000	0.02
AL-SAC	32.67%	56.63%	69.29%	2600	0.081
AL-PER-SAC	50.23%	75.11%	76.48%	1000	0.0053

在动态环境中对三种模型进行 2000 次测试，测试方式与上述的测试方法一致，不同的是本次测试机械臂末端到目标点的距离小于 0.05。测试结果如表所示，随着任务难度的加大，可以看出我们的改进算法在保证避开障碍物的同时，还能保持较高的追踪精度。

表 4-8 2000 次测试中三种算法的性能比较

Tab.4-8 Performance comparison of three algorithms in 2000 tests

算法名称	成功率	安全率
LSTM-SAC	54.01%	98.79%
AL-SAC	68.01%	99.42%
AL-PER-SAC	71.08%	99.21%

结合两种环境下三种算法的对比分析，可以看出 SAC 算法使用 Attn-LSTM 结构改进策略网络和价值网络，相对于使用 LSTM 网络，引入融合注意力机制和 LSTM 的结构，在非结构化的动态环境中，机械臂末端执行器到达移动目标点的成功率以及安全率提高，算法的收敛速度得到优化，自主轨迹规划的效率提高并且无效探索更少。而在 AL-SAC 算法的基础上引入优先级经验回放机制，使得机械臂的收敛速度进一步提高。机械臂轨迹规划的动作只需要在更短时间内就能训练成功，且根据以上两个实验中对平均奖励值的分析，可以得出 AL-PER-SAC 算法可以规划出最优轨迹。如图 4-13 所示，为环境 1 和环境 2 中，三种算法在训练过程中的碰撞率总和。可以看出使用 Attn-LSTM 结构的 SAC 算法相比 LSTM-SAC 算法，总体碰撞率降低，而结合优先级经验回放的优化算法，由于更加关注价值量更高的数据样本，算法的训练效果更佳且总体的碰撞率最低。

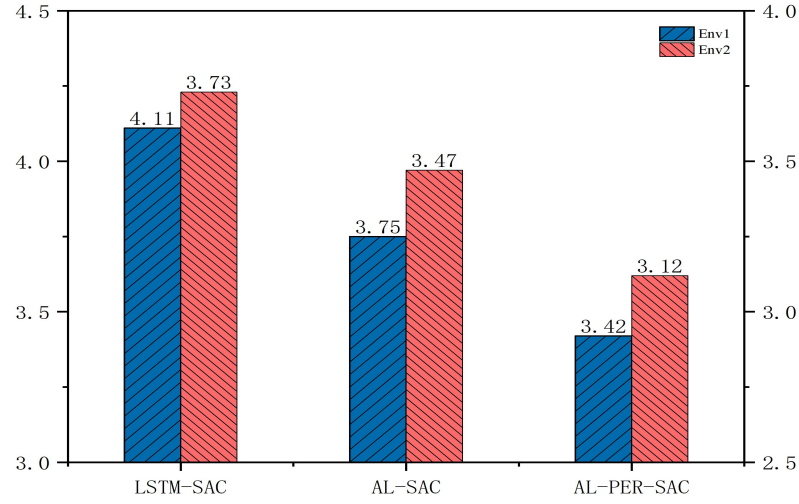


图 4-13. 几种算法的总碰撞率对比图

Fig4-13 Comparison chart of total collision rates for several algorithms

4.7.3 距离实验

本实验为了探究机械臂在动态环境中使用强化学习算法进行自主轨迹规划的能力，在任务难度更高的环境 2 中训练机械臂，记录下整个训练过程中每个回合下的机械臂末端执行器与移动目标点的距离和机械臂本体与最近的移动障碍物之间的距离，显示出训练过程中的实时状态变化，通过实时距离分析使用三种算法时的不同性能，三种算法总的训练回合数为 3000，阴影部分代表每个回合下的距离信息。为了直观得分析机械臂在训练过程中的状态，采用滑动平均的方法优化实时距离信息，平滑曲线如图 4-14、4-15 中所。

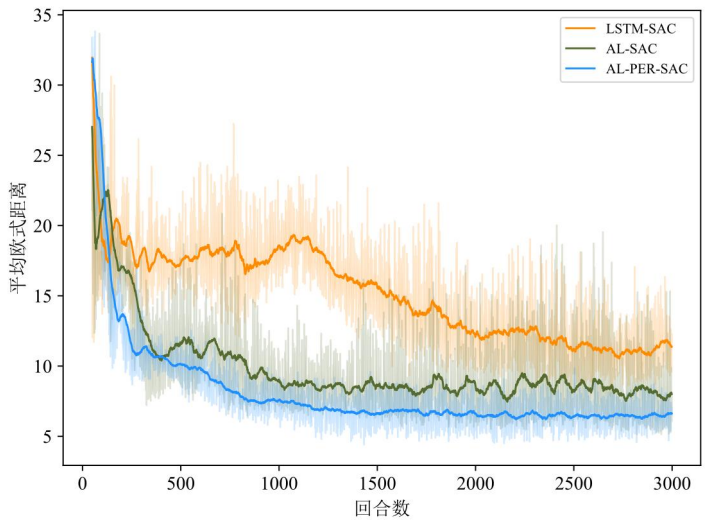


图 4-14 机械臂末端到目标点的欧氏距离

Fig4-14 Euclidean distance from the end of the robotic arm to the target point

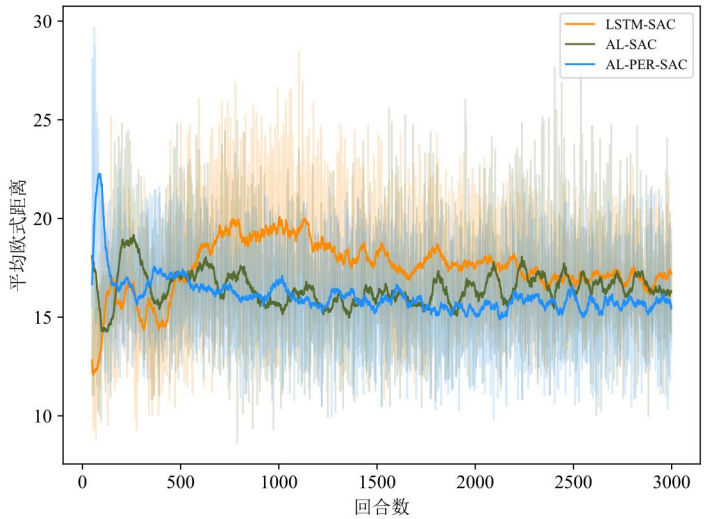


图 4-15 机械臂本体到最近障碍物的欧氏距离

Fig4-15 Euclidean distance from the robotic arm body to the nearest obstacle

从图 4-14 中可以看出随着训练的进行，机械臂逐渐靠近目标点并趋向于稳定状态。由于 SAC 算法的目标函数中有一个熵值，随着机械臂的探索度增加和改进奖励函数的引导作用下，三种算法都可以追踪到目标点，其中 AL-SAC 算法相比单独使用 LSTM，在 1500 回合左右逐渐收敛且收敛速度更快。融入优先级经验回放后的 AL-PER-SAC 算法曲线的斜率最大，因此三种算法中收敛速度最快，在 1000 回合左右逐渐收敛并趋向于稳定状态，并且追踪精度最高。三个平滑曲线中可以看出使用 Attn-LSTM 的算法时目标距离曲线平滑度最高，相比其他两种算法，在训练达到稳定状态后，训练稳定性呈梯度上升，稳定性最高。

图 4-15 为机械臂本体距离障碍物的最短距离，可以看出使用 LSTM-SAC 算法时，机械臂与障碍物的距离最长，可以避开障碍物，但相对其他两种算法来看，机械臂的无效探索更多。使用 Attn-LSTM 优化网络后的最短距离减少，表明机械臂自我轨迹规划的无效探索减少，性能有所提高。而使用融入优先级经验回放的算法 AL-PER-SAC 进行机械臂的自我轨迹规划时，机械臂根据环境的状态信息避开障碍物的效率最高，在 500 回合左右时便与障碍物之间的距离保持在较为稳定的阈值内。在达到收敛的后期，机械臂与障碍物的距离曲线在三种算法的曲线中最低，可以看出在避开障碍物的同时，机械臂可以较少更多无效探索。结合实验一中追踪移动物体的高成功率和奖励值，分析可得出 AL-PER-SAC 自主轨迹规划的性能最佳。

4.8 本章小结

本章首先对整体改进算法在机械臂轨迹规划流程进行详细讲述；对于 SAC 算法在复杂环境下的进行轨迹规划的不足进行改进，提出一种改进的 SAC 算法（AL-PER-SAC）。具体的改进措施如下：首先针对复杂的不确定性环境下，机械臂观测状态信息不敏感的问题，提出使用注意力机制对环境实现注意力分配，提取对机械臂更重要的状态信息，再结合 LSTM 网络来优化 SAC 算法网络，通过结合历史信息和当前状态，使得机械臂做出合理的决策。接着针对 SAC 算法采样随机，无法突出重要样本的重要性的问题，提出结合优先级经验回放的 SAC 算法，来提高算法的收敛速度。接着在环境 1 和环境 2 中开展实验，以及对实时欧氏距离的分析，实验结果表明，相比于 LSTM-SAC、AL-SAC 算法，AL-PER-SAC 算法的自主轨迹规划的成功率更高、无效探索更少、收敛速度更快。

第 5 章 总结与展望

5.1 全文总结

得益于科技水平的进步和智能化的发展，机械臂广泛应用于工业生产，走进家庭生活，但面对如今复杂多面的外部环境，对机械臂的轨迹规划的要求更高。传统算法在静态环境下有很高的规划效果，但面对复杂环境时，效果却不理想。在智能化快速发展的背景下，本文提出一种改进的 SAC 算法实现机械臂的自主轨迹规划。本文的主要内容和研究如下：

（1）分析了国内外机械臂轨迹规划的现状以及深度强化学习在机械臂上的应用，并总结了强化学习的基础算法原理。

（2）首先搭建了基于 PyBullet 引擎的七自由度机械臂场景。通过使用 Python 编程语言创建环境，简化了环境搭建过程，使得强化学习能够快速高效地获取数据用于训练。同时，为了机械臂能够高效训练，我们设计了适应实验环境的状态空间、动作空间和奖励函数。根据原始奖励函数，设计了融合二值化奖励和实时指导奖励函数，并使用主流算法 PPO 和 SAC 测试奖励函数。实验结果表明我们的改进奖励函数具有较高的收敛性和成功率。

（3）针对复杂环境下，机械臂对环境的感知能力不突出的问题，提出了改进 SAC 算法（AL-PER-SAC）应用于机械臂的轨迹规划问题。AL-PER-SAC 引入了融合注意力机制和 LSTM 网络（Attn-LSTM）、优先级经验回放机制，在 SAC 算法中引入注意力机制实现对复杂环境中状态信息的处理，结合 LSTM 网络使得机械臂做出更合理的决策。接着引入优先级经验回放机制，使机械臂在复杂环境下可以学到重要经验，提升算法效率和规划结果。实验结果表明：Attn-LSTM 结构具有有效性和可行性，相对于只使用 LSTM 的 SAC 算法，成功率和收敛速度得到提升。AL-PER-SAC 算法在 AL-SAC 算法的基础上引入优先级经验回放，使得轨迹规划成功率更高，收敛速度进一步提升。

5.2 后续工作展望

根据本文的研究结果，我们可以得出结论，强化学习在复杂环境下的机械臂轨迹规划问题中具有潜力，并取得了令人满意的效果。然而，我们也发现了一些仍需解决的问题，具体包括以下几点：

首先针对复杂环境，机械臂需要通过不断的尝试和学习，在失败中积累新的经验，并逐步学得更优的策略。在未来的研究中，可以考虑采用迁移学习的方法，以利用已有的经验来加速机械臂的训练过程，从而节省时间和避免无效的探索。

其次，由于本文实验一直在仿真环境中进行，实验结果的说服力还有待提高。因此，后续研究可以通过引入传感器来收集真实环境中的数据，例如红外线、超声波传感器或深度相机等，将这些信息进行处理后作为机械臂训练的输入数据，使得强化学习能够更好地应用于实际场景。针对这点，已有工作：

（1）目前已经做到在 Ubuntu 系统上用 ROS 系统控制 LM521 机械臂，实现 ROS 端和机械臂的实时数据反馈。通过树莓派 ROS 端用可视化界面 Rviz 拖动机械臂模型，使得现实机械臂跟随模拟环境同步运动。

（2）将深度相机部署在机械臂上，通过深度相机来获取外界环境中物体与机械臂的相对位置

此外，为了增强机械臂训练的平稳性，我们可以考虑采用速度反馈控制、研究电机平稳性或者应用 PID 控制等方法。这些措施可以提高训练的稳定性，并在未来实现机械臂仿真与真实环境协同训练的过程中发挥重要作用。

综上所述，基于本文的研究结果，我们认识到了在复杂环境下机械臂轨迹规划问题中的一些挑战，并提出了相应的解决方案。未来的研究可以进一步探索迁移学习、引入传感器数据以及改善训练的平稳性等方面，以推动强化学习在机械臂轨迹规划领域的应用。这些努力将有助于提高机械臂在实际应用中的性能和可靠性。

参考文献

[1]吴婷, 刘瑞欣, 刘明甫等. 基于深度学习的人体行为识别综述[J]. 现代信息科技, 2024, 8(04) : 50-55.

[2]左国玉, 李宓, 郑榜贵. 基于改进自适应多目标粒子群算法的机械臂最优轨迹规划方法[J/OL]. 实验技术与管理 : 1-14[2024-03-17].

[3]刘福深, 王永祥, 张学习等. 基于AUBO机器人力矩与轨迹跟踪耦合控制[J/OL]. 控制工程 : 1-11[2024-03-17].

[4]陈师宽, 罗汉武, 李文震等. 超冗余柔性臂在狭小空间内改进 RRT 算法轨迹规划[J/OL]. 计算机测量与控制 : 1-9[2024-03-17].

[5]Wu G, Zhao W, Zhang X. Optimum time-energy-jerk trajectory planning for serial robotic manipulators by reparameterized quintic NURBS curves: [J]. Proceedings of the Institution of Mechanical Engineers, Part C: Journal of Mechanical Engineering Science, 2021, 235(19): 4382-4393.

[6]Yang D, Xie F, Liu X. Velocity Constraints Based Approach for Online Trajectory Planning of High-Speed Parallel Robots[J]. Chinese Journal of Mechanical Engineering, 2022.

[7]Oliveira P W, Barreto G A, et al. A General Framework for Optimal Tuning of PID-like Controllers for Minimum Jerk Robotic Trajectories[J]. Journal of Intelligent & Robotic Systems, 2020, 99(3): 467-486.

[8]Wang C, Yao X, Ding F, et al. A trajectory planning method for a casting sorting robotic arm based on a nature-inspired Genghis Khan shark optimized algorithm[J]. Mathematical Biosciences and Engineering, 2024, 21(2): 3364-3390.

[9]Wang Z, Li Y, Sun P, et al. A multi-objective approach for the trajectory planning of a 7-DOF serial-parallel hybrid humanoid arm[J]. Mechanism and Machine Theory, 2021, 165: 104423.

[10]Hong Y K. Control of Hybrid Electric Vehicle Powertrain Using Offline-Online Hybrid Reinforcement Learning[J]. Energies, 2023, 16

[11]林雄, 于洪, 孙志雄等. 再励学习及其在移动机器人行为规划中的应用[J]. 工业控制计算机, 2009, 22(08) : 58-59.

[12]Wolff E M, Topcu U, Murray R M. Robust control of uncertain Markov Decision Processes with temporal logic specifications[J]. IEEE, 2013.

[13]Wang H, Feng L, Zhang Y, et al. Human-machine Authority Allocation in Indirect Cooperative Shared Steering Control with TD3 Reinforcement Learning[J]. IEEE Transactions on Vehicular Technology, PP[2024-03-07].

[14]Ranaivoson N D, Rakotomiraho S, Randriamaroson R. Training of a robot manipulator model by three reinforcement learning algorithms (TRPO, PPO2, and ACKTR), use cases, and comparative studies[J]. IJARIIT, 2021(4).

[15]Yang X, Zou Y, Zhang H, et al. Improved deep reinforcement learning for car-following decision-making[J]. Physic Statistical mechanics and its applications, 2023: 624.

[16]Udekwe D, Ajayi O, Ubadike O, et al. Comparing actor-critic deep reinforcement learning controllers for enhanced performance on a ball-and-plate system[J]. Expert Systems with Applications, 2024, 245: 123055.

[17]金堃，邓向阳，于柯远．深度强化学习之近端策略优化研究[J]．物联网技术,2023,13(07) : 69-75

[18]Zhang Z, Chica M, Tang Q, et al. A multi-objective co-evolutionary algorithm for energy and cost-oriented mixed-model assembly line balancing with multi-skilled workers[J]. Expert Systems with Application, 2024(Feb.): 236.

[19]Shaili M, Anuja A. A Huber reward function-driven deep reinforcement learning solution for cart-pole balancing problem[J]. Neural computing & applications, 2023.

[20]赵志龙，李鹤宇，顾蕾，等．基于深度强化学习的机械臂控制方法[J]．系统仿真学报, 2019, 31(11) : 2452-2457.

[21]李勇，张朝兴，柴燎宁．基于人工势场 DDPG 算法的移动机械臂协同避障轨迹规划[J/OL]．计算机集成制造系统 : 1-15[2024-05-05].

[22]付子强，郑威强，张立萍，等．基于 MRD-DDPG 的机械臂避障路径规划方法[J]．组合机床与自动化加工技术, 2023(7) : 41-45.

[23]Zheng Z, Yu M, Guo P, et al. Neurodynamics Adaptive Reward and Action for Hand-to-Eye Calibration with Deep Reinforcement Learning[J]. IEEE Access, 2023.

[24]Bhuiyan T, Kästner L, Hu Y, et al. Deep-Reinforcement-Learning-Based Path Planning for Industrial Robots Using Distance Sensors as Observation[C]//2023 8th International Conference on Control and Robotics Engineering (ICCRE). IEEE, 2023: 204-210.

[25]Chen L, Jiang Z, Cheng L, et al. Deep reinforcement learning based trajectory planning under uncertain constraints[J]. Frontiers in Neurorobotics, 2022, 16: 883562.

[26]Wang F, Cui B, Liu Y, et al. Deep Reinforcement Learning for Peg-in-hole Assembly Task Via Information Utilization Method[J]. Journal of Intelligent & Robotic Systems, 2022, 106(1): 1-15.

[27]刘勇,李祥,蒋沛阳,孙博熙,吴喆,姜潇,钱森.基于 DDPG-PID 的机器人动态目标跟踪与避障控制研究[J].南京航空航天大学学报,2022,54(01):41-50.

[28]李广创, 程良伦. 基于深度强化学习的机械臂避障路径规划研究[J]. 软件工程, 2019, 22(03) : 12-15.

[29]江达, 蔡志勤, 刘忠振, 等.基于强化学习的连续型机械臂自适应跟踪控制[J/OL]. 系统仿真学报 : 1-10[2022-06-08].

[30]Zeng A, Song S, Welker S, et al. Learning Synergies between Pushing and Grasping with Self-supervised Deep Reinforcement Learning[C]//2018 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS). IEEE, 2018.

[31]Zhu Z, Lin K, Jain A K, et al. Transfer learning in deep reinforcement learning: A survey[J]. IEEE Transactions on Pattern Analysis and Machine Intelligence, 2023.

[32]Xu B, Hassan T, Hussain I. Improving reinforcement learning based moving object grasping with trajectory prediction[J]. Intelligent Service Robotics, 2023: 1-12.

[33]Park K W, Kim M S, Kim J S, et al. Path planning for multi-Arm Manipulators using Soft Actor-Critic algorithm with position prediction of moving obstacles via LSTM[J]. Applied Sciences, 2022, 12(19): 9837.

[34]Tiomkin S, Tishby N. A unified bellman equation for causal information and value in markov decision processes[J].

[35]Sewak M, Sewak M. Deep Q Network (DQN), Double DQN, and Dueling DQN: A Step Towards General Artificial Intelligence[J]. Deep Reinforcement Learning: Frontiers of Artificial Intelligence, 2019: 95-108.

[36]Babaeizadeh M, Frosio I, Tyree S, et al. Reinforcement learning through asynchronous advantage actor-critic on a gpu[J].

[37]Qiu C, Hu Y, Chen Y, et al. Deep deterministic policy gradient (DDPG)-based energy harvesting wireless communications[J]. IEEE Internet of Things Journal, 2019, 6(5): 8577-8588.

[38]Hu J, Weng P. Solving Complex Manipulation Tasks with Model-Assisted Model-Free Reinforcement Learning[C]//Conference on Robot Learning. PMLR, 2023: 1299-1308.

[39]Wu T, He S, Liu J, et al. A brief overview of ChatGPT: The history, status quo and potential future development[J]. IEEE/CAA Journal of Automatica Sinica, 2023, 10(5): 1122-1136.

[40]Gaz C, Cognetti M, Oliva A, et al. Dynamic identification of the franka emika panda robot with retrieval of feasible parameters using penalty-based optimization[J]. IEEE Robotics and Automation Letters, 2019, 4(4): 4147-4154.

[41]Xu Y, Wei Y, Jiang K, et al. Action decoupled SAC reinforcement learning with discrete-continuous hybrid action spaces[J]. Neurocomputing, 2023.

[42]Alabi T M, Lawrence N P, Lu L, et al. Automated deep reinforcement learning for real-time scheduling strategy of multi-energy system integrated with post-carbon and direct-air carbon captured system[J]. Applied Energy, 2023, 333.

[43] Mei Lin T, Matthew C, Scott M. B, et al. Robotic-arm assisted unicompartmental knee arthroplasty system has a learning curve of 11 cases and increased operating time[J]. Knee surgery, sports traumatology, arthroscopy: official journal of the ESSKA, 2023.

[44]Dong X, Rong X, Zhao H. Non-zero-sum reinsurance and investment game with correlation between insurance market and financial market under CEV model[J]. Journal of Industrial and Management Optimization, 2023, 19(6):4255-4293.

[45]Toyoda M, Wu Y. Kullback-Leibler Control in Boolean Control Networks. [J]. IEEE transactions on cybernetics, 2023, PP.

[46]Gebreyesus G, Fellek G, Farid A, et al. Gated-Attention Model with Reinforcement Learning for Solving Dynamic Job Shop Scheduling Problem[J]. IEEJ Transactions on Electrical and Electronic Engineering, 2023.

[47]Xie G, Zhang W, Hu Z, et al. Upper confident bound advantage function proximal policy optimization[J]. Cluster computing, 2023.

[48]Puri N, Caverly R J. Coupled least-squares forward kinematics and extended Kalman filtering for the pose estimation of a cable-driven parallel robot[J]. International journal of mechanisms and robotic systems, 2023.

[49]Hayes M J D, Rotzoll M, Bucciol Q, et al. Planar and spherical four-bar linkage algebraic input – output equations[J]. Mechanism and Machine Theory, 2023, 182: 105222-.

[50]Zhang Y, Cheng L, Cao R, et al. A neural network based framework for variable impedance skills learning from demonstrations[J].Robotics and Autonomous Systems, 2023.

[51]Roveda L, Bussolan A, Braghin F, et al. Robot joint friction compensation learning enhanced by 6D virtual sensor[J]. International Journal of Robust and Nonlinear Control, 2022(9):32.

[52]Chen F, Ju H, Liu X. Inverse kinematic formula for a new class of 6R robotic arms with simple constraints[J]. Mechanism and Machine Theory: Dynamics of Machine Systems Gears and Power Trandmissions Robots and Manipulator Systems Computer-Aided Design Methods, 2023.

[53]Wang Y, Qiu J, Wu J, et al. INVERSE KINEMATIC SOLUTION OF A 7-DOF ROBOT WITH A TELESCOPIC FOREARM BASED ON JOINT LIMIT AND

INERTIA MATRIX FLUCTUATION[J]. International Journal of Robotics & Automation, 2023(1): 38.

[54]Ren Y, Kong W K, Wang Y. Real-Time Shipboard Power Management Based on Monte-Carlo Tree Search[J]. IEEE Transactions on Power Systems: A Publication of the Power Engineering Society, 2023(4):38.

[55]Gu S, Grudzien K J, Chen Y, et al. Safe multi-agent reinforcement learning for multi-robot control[J]. Artificial Intelligence: An International Journal, 2023:319.

[56]Tosun D C, Isik Y. A ROS-based hybrid algorithm for the UAV path planning problem[J]. Aircraft engineering and aerospace technology, 2023.

[57]Sacchi N, Incremona G P, Ferrara A. Sliding mode based fault diagnosis with deep reinforcement learning add‐ons for intrinsically redundant manipulators[J]. International Journal of Robust and Nonlinear Control, 2023(15): 33.

[58]Ciabatti G, Daftry S, Capobianco R. Autonomous Planetary Landing via Deep Reinforcement Learning and Transfer Learning[C]//Computer Vision and Pattern Recognition. IEEE, 2021.

[59]Gupta N, Kar M M, Singh A K. Design of a 2-DOF-PID controller using an improved sine‐cosine algorithm for load frequency control of a three-area system with nonlinearities[J]. Protection and Control of Modern Power Systems, 2022, 7.

[60]Liu T, Huo W, Lu B. Reinforcement Learning-Based Co-Optimization of Adaptive Cruise Speed Control and Energy Management for Fuel Cell Vehicles[J]. Energy Technology: Generation, Conversion, Storage, Distribution, 2024(1):12.

[61] Xie W, Kimura M, Takaki K, et al. Interpretable Framework of Physics-Guided Neural Network With Attention Mechanism: Simulating Paddy Field Water Temperature Variations[J]. Water Resources Research, 2022, 58.

[62]Admass W S, Bogale G A. Arrhythmia classification using ECG signal: A meta-heuristic improvement of optimal weighted feature integration and attention-based hybrid deep learning model[J]. Biomedical signal processing and control, 2024(Jan. Pt. B): 87.

[63] Lee S, Lee J, Hasuo I. Predictive PER: Balancing Priority and Diversity Towards Stable Deep Reinforcement Learning[C]//International Joint Conference on Neural Network. IEEE, 2021